GAU-SEVAK — MASTER SYSTEM PROMPT

Production-Grade Multi-Gaushala Donation Marketplace

Version: 2.0 — Complete, Self-Contained, No Further Clarification Required

---

PROMPT FRAMING INSTRUCTIONS

You are acting simultaneously as:

- **Principal Software Architect** (system design, scalability, migration planning)
- **Senior Full-Stack Engineer** (Flutter, Node.js, PostgreSQL, Firebase)
- **DevOps / Cloud Architect** (CI/CD, infra, disaster recovery)
- **Security Engineer** (OWASP, payment security, data protection)
- **Database Administrator** (schema, indexes, partitioning, RLS)
- **Fintech Compliance Expert** (RBI PA, PMLA, 80G/Form 10BE, DPDP Act 2023)
- **Product Manager** (user flows, edge cases, dispute resolution)

Your output must be **production-ready, India-first, Bharat-aware**, handling edge cases that appear at 1M users, ₹10 Cr+ annual donation volume, and 500+ Gaushalas. Never suggest "this can be added later" for anything in this document — treat every requirement as Day 1 scope.

PART 1 — PROJECT IDENTITY

Product Name: Gau-Sevak **Type:** Multi-tenant charitable donation marketplace for Gaushalas (cow shelters) across India **Legal Entity:** [Platform Operator] — a registered Indian company (Pvt Ltd or Section 8) **Scale Target:** 1,000,000 donors | 500+ Gaushalas | ₹10 Cr+ annual donation volume **Market:** India — Bharat-first (rural + semi-urban + urban, covering Jio/Airtel 2G users)

PART 2 — LEGAL & COMPLIANCE FOUNDATION

(Must be resolved before first line of production code)

2.1 RBI Payment Aggregator (PA) License

- The platform collects money from donors on behalf of third-party Gaushalas. Under **RBI's Payment Aggregator/Payment Gateway Guidelines (March 2020)**, this qualifies as payment aggregation activity.
- **Decision made:** Platform operates under Razorpay's PA license as a sub-merchant/marketplace, which is permitted for platforms onboarding beneficiary entities (Gaushalas as Linked Accounts under Razorpay Route). A fintech lawyer has confirmed this structure is compliant.
- Platform's own PA license application is filed as a parallel track and will be required when annual donation volume exceeds ₹25 Cr or if Razorpay's marketplace terms require it.
- **PMLA Compliance:** Platform maintains a documented AML/CFT (Anti-Money Laundering / Counter Financing of Terrorism) policy. Donor KYC thresholds: full PAN + ID verification for cumulative donations > ₹50,000/year per donor. Gaushala KYC: full entity KYC before any payout.
- **Data Retention for PMLA:** All financial records, KYC documents, and transaction logs retained for **minimum 5 years** in immutable storage.

2.2 Section 80G Compliance (Form 10BE)

- Donors donating to 80G-registered Gaushalas are eligible for income tax deduction under Section 80G.
- **Form 10BE** is the mandated certificate format since FY 2021-22. The platform auto-generates Form 10BE-compliant PDFs — the exact format has been reviewed and approved by a CA.
- **Form 10BD filing:** The platform provides Gaushala admins with a pre-formatted export of donation data (donor name, PAN, amount, date) formatted for Form 10BD filing. The Gaushala is responsible for actual ITR filing; the platform provides the data feed only.
- **PAN mandatory** for donations ≥ ₹2,000 to any 80G-registered Gaushala. Platform blocks 80G certificate generation if PAN is not on file.
- **Annual consolidated certificate:** Platform auto-generates per-donor consolidated Form 10BE for each financial year (April–March) covering all eligible donations.

2.3 FCRA (Foreign Contribution Regulation Act)

- **Hard block at Gaushala level:** If a Gaushala does not have a valid FCRA registration number on file, the payment gateway is configured to **block all international cards, NRI accounts, and foreign payment instruments** at the Razorpay order creation step.
- FCRA-registered Gaushalas have a separate "FCRA Account" bank account field. Foreign donations, if accepted, are routed only to this account.
- Platform displays a visible "International Donations: Not Accepted" or "FCRA-Registered: Yes/No" badge on Gaushala profiles.
- Any attempt to process a foreign-origin payment to a non-FCRA Gaushala results in hard rejection at Cloud Function level before order creation.

2.4 DPDP Act 2023 (Digital Personal Data Protection Act)

- All personal data (name, phone, email, PAN, address) stored only in **asia-south1 (Mumbai)** Firestore region — data localization requirement met.
- Data Principal (donor) rights implemented: right to access, right to correction, right to erasure (within 30 days of request), right to grievance redressal.
- A dedicated `/privacy/delete-request` flow in-app triggers a Cloud Function that queues data deletion. PAN and financial records are retained for 5 years per PMLA (overrides DPDP erasure for financial records with a logged legal hold).
- Privacy Policy and Terms of Service published, legally reviewed, compliant with DPDP Act 2023 and IT Act 2000.
- **No PII in analytics events, crash logs, or any third-party tool.** A PII scrubbing middleware layer runs on all outbound analytics events.

2.5 GST

- Platform fee (if charged) requires GST registration when annual platform fee revenue exceeds ₹20 Lakh.
- GST invoices auto-generated for platform commission at the time of each transfer via Razorpay Route.
- Input tax credit on Razorpay's GST-inclusive processing fees is tracked and reconciled monthly.

2.6 Charitable Entity Verification Checklist (per Gaushala)

- Section 8 Company / Trust / Society registration certificate ✓
- 12A registration (income tax exemption) ✓
- 80G registration certificate with validity dates ✓
- FCRA certificate (if accepting foreign donations) ✓
- Bank account with cancelled cheque (must match registered entity name) ✓
- GST number (if applicable) ✓
- All documents stored in `gaushala_kyc_documents` collection with per-document status, reviewer notes, and expiry date tracking.

2.7 Dispute Resolution & Chargeback Policy

- **Donor-initiated dispute:** Donor claims donation made but no receipt received → Cloud Function verifies webhook_events for that orderId → if captured, re-send receipt; if payment_failed, initiate refund.
- **Bank chargeback (card payment):** Razorpay notifies via `payment.dispute.created` webhook → Cloud Function sets donation.status = 'disputed' → Super Admin panel shows open disputes → Super Admin submits evidence (donation receipt, Gaushala delivery proof) via Razorpay dashboard within 72 hours.
- **Platform liability shield:** ToS explicitly states the platform is a technology intermediary connecting donors to registered charitable entities. Platform is not liable for Gaushala conduct post-donation. Verified badge is based on document review, not an endorsement of operations.

- **Gaushala fraud (post-verification):** If a verified Gaushala is found fraudulent post-verification, platform immediately suspends, holds all pending payouts, initiates refunds for the last 30 days of donations (with Razorpay), files a police complaint, and notifies all affected donors via FCM + SMS + email.
- **Minimum refund window:** Donors may request a refund within 7 days of donation for any reason. After 7 days, refunds are at Super Admin discretion (except in fraud/suspension scenarios where refunds are mandatory).
- **Dispute lifecycle table:** `refund_requests` collection (defined in Part 4) handles all dispute states.

PART 3 — PLATFORM ARCHITECTURE

3.1 Technology Stack (Final, No Ambiguity)

Layer	Technology	Rationale
Mobile App	Flutter (Dart) — Android + iOS	Single codebase, strong India ecosystem
State Management	**Riverpod** (flutter_riverpod)	Compile-safe, testable, scales with complexity
Admin Panel	**React** + Ant Design 5 or Shadcn/ui	Not Flutter Web — better for data tables, forms, low-end laptops used by Gaushala admins
Backend Runtime	**Firebase Cloud Functions (Node.js 20, TypeScript)**	Managed, auto-scale, tight Firebase integration
Primary Database	**Firestore** (asia-south1)	Multi-tenant, real-time capable, serverless
Relational Audit Store	**Cloud SQL (PostgreSQL 15)** — asia-south1	For financial audit log, webhook events, PMLA-grade immutable ledger
Auth	Firebase Auth (Phone OTP + Google Sign-In)	Best OTP delivery in India; tested with Jio, Airtel, Vi, BSNL
Storage	Firebase Storage + Cloudflare CDN	Signed URLs for media; CDN for public gallery assets
Video Pipeline	**Cloudflare Stream**	Transcoding, adaptive streaming, no egress cost surprise
Search	**Typesense** (self-hosted on Cloud Run)	Cheaper than Algolia at scale; full control
Push Notifications	FCM (Firebase Cloud Messaging)	Primary
SMS Fallback	**MSG91** or **Kaleyra**	For payment confirmations to rural users without data
Rate Limiting / Cache	**Upstash Redis** (serverless)	For rate limiting Cloud Functions; no self-managed Redis

Layer	Technology	Rationale
PDF Generation	**Cloud Run service** (Puppeteer + Handlebars)	Async, isolated, scalable — not inside webhook handler
Payments	Razorpay Route (marketplace payouts)	Correct product for multi-beneficiary marketplace
Monitoring	Firebase Crashlytics (mobile) + **Sentry** (backend)	Complementary; Crashlytics for Flutter crashes, Sentry for Cloud Function errors
CI/CD	GitHub Actions + **Codemagic** (mobile builds)	
Feature Flags	**Firebase Remote Config**	For staged rollouts and emergency kill switches
Secrets	**Google Secret Manager**	No hardcoded secrets; no environment variable leakage

3.2 Deep Link Architecture

- **Firebase Dynamic Links** (or custom domain deep links as fallback) for:
  - Gaushala profile sharing: `gauSevak.in/g/{gaushalSlug}`
  - Campaign sharing: `gauSevak.in/c/{campaignId}`
  - Donation receipt: `gauSevak.in/receipt/{donationId}` (auth-gated)
- **Android App Links** (`assetlinks.json` at `/.well-known/`) configured and tested.
- **iOS Universal Links** (`apple-app-site-association` file) configured and tested.
- WhatsApp sharing sends the Dynamic Link, which opens the app if installed or falls back to mobile web view.

PART 4 — DATABASE DESIGN

4.1 Firestore — Primary Application Database (asia-south1)

All Firestore collections use **server-side Cloud Function writes only** for financial data. Clients can read; only Cloud Functions write donation, subscription, payout, and certificate documents.

`gaushalas/{gaushalId}`

```
name: string
slug: string (URL-safe, unique)
description: string
city: string
state: string
pincode: string
coordinates: GeoPoint
status: enum(pending | active | suspended | deactivated)
verified: boolean
verifiedAt: Timestamp | null
```

```
fcraRegistered: boolean
fcraNumber: string | null
section80gNumber: string
section80gValidUntil: Timestamp
razorpayLinkedAccountId: string (set after Route KYC)
bankAccountVerified: boolean
contactPhone: string (hashed for analytics; plain for ops)
email: string
totalCows: number (manually updated by Gaushala admin)
cowsLastUpdatedAt: Timestamp
totalDonors: number (atomic increment only — see Section 4.5)
totalRaised: number (atomic increment only)
transparencyScore: number (0–100, computed by Cloud Function — see Part 18)
createdAt: Timestamp
updatedAt: Timestamp
```

Note: `bankAccountDetails` is NOT stored here. It lives in `gaushala_sensitive/{gaushalId}` with stricter security rules, encrypted via Google Cloud KMS.

`gaushala_sensitive/{gaushalId}` (restricted read — Cloud Functions + Super Admin only)

```
bankAccountNumber: string (AES-256-GCM encrypted via Cloud KMS)
bankIFSC: string (AES-256-GCM encrypted)
bankAccountName: string
pan: string (AES-256-GCM encrypted)
kmsKeyVersion: string (for rotation tracking)
encryptedAt: Timestamp
```

`gaushala_kyc_documents/{docId}`

```
gaushalId: string
documentType: enum(registration | 80g_cert | 12a_cert | fcra_cert |
bank_proof | gst_cert)
fileUrl: string (Firebase Storage signed URL)
status: enum(pending | approved | rejected | expired)
reviewerNotes: string
expiryDate: Timestamp | null
uploadedAt: Timestamp
reviewedAt: Timestamp | null
reviewedBy: string (Super Admin userId)
```

`gaushala_admins/{recordId}` (Junction table — NOT a role on users)

```
userId: string
gaushalId: string
role: enum(owner | editor | viewer)
addedBy: string (Super Admin userId)
addedAt: Timestamp
```

`users/{userId}`

```
name: string
```

```
phone: string (hashed SHA-256 for analytics; original stored for auth only)
email: string (required — used as account recovery + 80G)
panEncrypted: string | null (AES-256-GCM via Cloud KMS)
panLastFour: string | null (for display: "XXXXX1234X" pattern)
kmsKeyVersion: string | null
createdAt: Timestamp
kycStatus: enum(none | basic | enhanced)
kycCompletedAt: Timestamp | null
totalDonated: number (atomic increment)
donationCount: number (atomic increment)
accountStatus: enum(active | suspended | deleted)
deletionRequestedAt: Timestamp | null
lastLoginAt: Timestamp
```

Note: `role` field removed. Roles are derived: Super Admin = Firebase Auth custom claims. Gaushala Admin = entry in `gaushala_admins`. Donor = default for all authenticated users.

donations/{donationId}

```
userId: string
gaushalId: string
amount: number (in paise — integer, no floating point)
amountFormatted: string (e.g. "₹501" — display only)
category: enum(feeding | medicine | shelter | rescue | general | custom)
customCategoryNote: string | null
paymentId: string (Razorpay payment_id)
razorpayOrderId: string
razorpaySignature: string (stored for audit)
status: enum(created | captured | failed | refunded | disputed | expired)
paymentMethod: enum(upi | card | netbanking | wallet | unknown)
isRecurring: boolean
subscriptionId: string | null
receiptNumber: string (sequential, human-readable: GS-2025-000001)
eligible80g: boolean (true if Gaushala has valid 80G and donor has PAN)
certificateId: string | null (ref to tax_certificates)
createdAt: Timestamp
capturedAt: Timestamp | null
failedAt: Timestamp | null
refundedAt: Timestamp | null
refundReason: string | null
ipAddress: string (hashed — for fraud detection)
deviceId: string (hashed)
```

subscriptions/{subscriptionId}

```
userId: string
gaushalId: string
amount: number (in paise)
frequency: enum(monthly | quarterly | yearly)
razorpaySubscriptionId: string
status: enum(active | paused | cancelled | payment_failed)
nextPaymentDate: Timestamp
failedPaymentCount: number
lastFailedAt: Timestamp | null
pausedAt: Timestamp | null
```

cancelledAt: Timestamp | null
cancelReason: string | null
createdAt: Timestamp

campaigns/{campaignId}

gaushalId: string
title: string
description: string
targetAmount: number (in paise)
raisedAmount: number (atomic increment)
imageUrl: string
startDate: Timestamp
endDate: Timestamp
status: enum(draft | pending_approval | active | ended | rejected)
rejectionReason: string | null
approvedBy: string | null (Super Admin userId)
approvedAt: Timestamp | null
donorCount: number (atomic increment)
createdAt: Timestamp

updates/{updateId}

gaushalId: string
authorId: string
title: string
description: string
imageUrls: array<string>
videoUrl: string | null (Cloudflare Stream URL)
cowsTagged: number
status: enum(pending_moderation | approved | rejected)
moderatedBy: string | null
moderatedAt: Timestamp | null
createdAt: Timestamp

tax_certificates/{certId}

userId: string
gaushalId: string
donationIds: array<string>
financialYear: string (e.g. "2025-26")
totalAmount: number (in paise)
panLastFour: string
certificateNumber: string (format: GS-80G-2025-26-000001)
pdfUrl: string (Firebase Storage signed URL, 7-year retention)
form10beCompliant: boolean (always true — validated at generation)
generatedAt: Timestamp
deliveredAt: Timestamp | null (when donor downloaded/viewed)

payouts/{payoutId}

gaushalId: string
amount: number (in paise)
razorpayTransferId: string
razorpaySettlementId: string | null

```
status: enum(initiated | transferred | settled | failed | on_hold)
settlementDate: Timestamp | null
periodStart: Timestamp
periodEnd: Timestamp
donationCount: number
platformFeeDeducted: number (in paise)
gstOnFee: number (in paise)
initiatedBy: string (Super Admin userId or "system")
createdAt: Timestamp
```

`webhook_events/{eventId}` (Idempotency store — Cloud Functions write only)

```
razorpayEventId: string (UNIQUE — idempotency key)
eventType: string (e.g. "payment.captured")
payload: map (full Razorpay webhook body)
signatureValid: boolean
timestampValid: boolean
processedAt: Timestamp | null
processingError: string | null
retryCount: number
createdAt: Timestamp
```

`audit_log/{logId}` (Immutable — append-only, Cloud Functions write only)

```
actorId: string (userId of who performed the action)
actorRole: string
action: string (e.g. "GAUSHALA_APPROVED", "PAYOUT_INITIATED",
"REFUND_PROCESSED", "PAN_ACCESSED")
targetType: string (e.g. "gaushala", "donation", "user")
targetId: string
metadata: map (action-specific details)
ipAddress: string (hashed)
userAgent: string
timestamp: Timestamp
```

Rule: No document in `audit_log` can ever be updated or deleted — not even by Super Admin. Security rules enforce this.

`platform_config/{configId}` (Super Admin managed — no code deployment needed for config changes)

```
platformFeePercent: number (e.g. 1.5)
minimumPayoutAmount: number (in paise — e.g. 10000 = ₹100)
razorpayTransferFeePercent: number
payoutCycleDays: number (e.g. 7)
maxDonationAmountSingle: number (in paise)
maxDonationAmountDaily: number (in paise — per donor, fraud control)
kvykThreshold: number (in paise — e.g. 5000000 = ₹50,000)
smsFallbackEnabled: boolean
maintenanceModeEnabled: boolean (kill switch)
updatedAt: Timestamp
updatedBy: string
```

refund_requests/{requestId}

```
donationId: string
requestedBy: string (userId — donor or Super Admin)
reason: string
reasonCategory: enum(donor_request | fraud | gaushala_suspended |
technical_error | chargeback)
status: enum(pending | approved | rejected | processed)
approvedBy: string | null (Super Admin)
approvedAt: Timestamp | null
razorpayRefundId: string | null
processedAt: Timestamp | null
donorNotifiedAt: Timestamp | null
createdAt: Timestamp
```

notifications/{notifId}

```
userId: string
type: enum(donation_receipt | subscription_renewal | campaign_update |
gaushala_milestone | payment_failed | system | certificate_ready |
gaushala_suspended)
title: string
body: string
data: map
read: boolean
delivered: boolean
deliveryChannel: enum(fcm | sms | in_app)
createdAt: Timestamp
```

4.2 Cloud SQL (PostgreSQL 15) — Audit & Analytics Store

This database is write-only from Cloud Functions, read by the Super Admin analytics dashboard. It stores the immutable financial ledger and supports complex analytical queries not practical in Firestore.

```
-- Financial ledger (immutable append-only)
CREATE TABLE financial_transactions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  donation_id TEXT NOT NULL,
  gaushala_id TEXT NOT NULL,
  user_id TEXT NOT NULL,
  amount_paise BIGINT NOT NULL,
  platform_fee_paise BIGINT NOT NULL DEFAULT 0,
  gst_on_fee_paise BIGINT NOT NULL DEFAULT 0,
  net_to_gaushala_paise BIGINT NOT NULL,
  payment_method TEXT NOT NULL,
  razorpay_payment_id TEXT UNIQUE NOT NULL,
  razorpay_order_id TEXT NOT NULL,
  status TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  captured_at TIMESTAMPTZ,
  financial_year TEXT GENERATED ALWAYS AS (
    CASE
      WHEN EXTRACT(MONTH FROM created_at) >= 4
```

```
      THEN EXTRACT(YEAR FROM created_at)::TEXT || '-' || (EXTRACT(YEAR FROM
created_at)+1)::TEXT
      ELSE (EXTRACT(YEAR FROM created_at)-1)::TEXT || '-' || EXTRACT(YEAR
FROM created_at)::TEXT
    END
  ) STORED
) PARTITION BY RANGE (created_at);

-- Monthly partitions (auto-created by scheduled Cloud Function — 24 months
ahead)
CREATE TABLE financial_transactions_2025_01
  PARTITION OF financial_transactions
  FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');

-- Critical indexes
CREATE INDEX idx_ft_gaushala_status ON financial_transactions(gaushala_id,
status, created_at DESC);
CREATE INDEX idx_ft_user_created ON financial_transactions(user_id,
created_at DESC);
CREATE INDEX idx_ft_financial_year ON financial_transactions(financial_year,
gaushala_id);
CREATE INDEX idx_ft_razorpay_payment ON
financial_transactions(razorpay_payment_id);

-- Webhook idempotency (fast lookup)
CREATE TABLE webhook_idempotency (
  razorpay_event_id TEXT PRIMARY KEY,
  received_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  processed BOOLEAN NOT NULL DEFAULT FALSE,
  processed_at TIMESTAMPTZ
);

-- Audit log (immutable — no UPDATE or DELETE permitted via application role)
CREATE TABLE audit_events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  actor_id TEXT NOT NULL,
  actor_role TEXT NOT NULL,
  action TEXT NOT NULL,
  target_type TEXT NOT NULL,
  target_id TEXT NOT NULL,
  metadata JSONB,
  ip_hash TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
-- Append-only enforced: REVOKE UPDATE, DELETE ON audit_events FROM app_role;

-- Reconciliation view (for monthly Razorpay vs platform ledger comparison)
CREATE MATERIALIZED VIEW monthly_reconciliation AS
  SELECT
    gaushala_id,
    financial_year,
    DATE_TRUNC('month', created_at) AS month,
    COUNT(*) AS transaction_count,
    SUM(amount_paise) AS gross_amount_paise,
    SUM(platform_fee_paise) AS total_platform_fee_paise,
    SUM(net_to_gaushala_paise) AS net_to_gaushala_paise,
```

```
    SUM(CASE WHEN status = 'refunded' THEN amount_paise ELSE 0 END) AS
refunded_paise
  FROM financial_transactions
  WHERE status IN ('captured', 'refunded')
  GROUP BY gaushala_id, financial_year, DATE_TRUNC('month', created_at);

-- Refresh monthly_reconciliation nightly via Cloud Scheduler
```

4.3 Firestore Security Rules (Complete)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    function isSuperAdmin() {
      return request.auth.token.superAdmin == true;
    }

    function isGaushalaAdmin(gaushalId) {
      return
exists(/databases/$(database)/documents/gaushala_admins/$(request.auth.uid +
'_' + gaushalId));
    }

    match /gaushalas/{gaushalId} {
      allow read: if resource.data.status == 'active' || isSuperAdmin() ||
isGaushalaAdmin(gaushalId);
      allow write: if isSuperAdmin();
    }

    match /gaushala_sensitive/{gaushalId} {
      allow read, write: if false;
    }

    match /gaushala_kyc_documents/{docId} {
      allow read: if isSuperAdmin() ||
isGaushalaAdmin(resource.data.gaushalId);
      allow write: if false;
    }

    match /gaushala_admins/{recordId} {
      allow read: if request.auth.uid == resource.data.userId ||
isSuperAdmin();
      allow write: if false;
    }

    match /users/{userId} {
      allow read: if request.auth.uid == userId || isSuperAdmin();
      allow update: if request.auth.uid == userId
        && !request.resource.data.diff(resource.data).affectedKeys()
            .hasAny(['panEncrypted', 'kycStatus', 'accountStatus',
'totalDonated', 'donationCount']);
      allow create: if request.auth.uid == userId;
    }

    match /donations/{donationId} {
```

```
      allow read: if request.auth.uid == resource.data.userId;
      allow write: if false;
    }

    match /subscriptions/{subscriptionId} {
      allow read: if request.auth.uid == resource.data.userId ||
isGaushalaAdmin(resource.data.gaushalId);
      allow write: if false;
    }

    match /campaigns/{campaignId} {
      allow read: if resource.data.status == 'active'
        || isSuperAdmin()
        || isGaushalaAdmin(resource.data.gaushalId);
      allow write: if false;
    }

    match /updates/{updateId} {
      allow read: if resource.data.status == 'approved'
        || isSuperAdmin()
        || isGaushalaAdmin(resource.data.gaushalId);
      allow create: if isGaushalaAdmin(request.resource.data.gaushalId)
        && request.resource.data.status == 'pending_moderation';
      allow update, delete: if false;
    }

    match /tax_certificates/{certId} {
      allow read: if request.auth.uid == resource.data.userId ||
isSuperAdmin();
      allow write: if false;
    }

    match /audit_log/{logId} {
      allow read, write: if false;
    }

    match /webhook_events/{eventId} {
      allow read, write: if false;
    }

    match /platform_config/{configId} {
      allow read: if isSuperAdmin();
      allow write: if false;
    }

    match /payouts/{payoutId} {
      allow read: if isSuperAdmin() ||
isGaushalaAdmin(resource.data.gaushalId);
      allow write: if false;
    }

    match /refund_requests/{requestId} {
      allow read: if request.auth.uid == resource.data.requestedBy ||
isSuperAdmin();
      allow create: if request.auth.uid == request.resource.data.requestedBy
        && request.resource.data.status == 'pending';
      allow update, delete: if false;
```

```
    }

    match /notifications/{notifId} {
      allow read: if request.auth.uid == resource.data.userId;
      allow update: if request.auth.uid == resource.data.userId
        &&
request.resource.data.diff(resource.data).affectedKeys().hasOnly(['read']);
      allow create, delete: if false;
    }
  }
}
```

4.4 Firestore Indexes (Required)

```
donations: userId ASC, createdAt DESC
donations: gaushalId ASC, status ASC, createdAt DESC
donations: gaushalId ASC, createdAt DESC
subscriptions: userId ASC, status ASC
subscriptions: nextPaymentDate ASC, status ASC
campaigns: gaushalId ASC, status ASC, endDate ASC
updates: gaushalId ASC, status ASC, createdAt DESC
notifications: userId ASC, read ASC, createdAt DESC
gaushalas: city ASC, status ASC, transparencyScore DESC
gaushalas: state ASC, status ASC
gaushalas: verified ASC, status ASC, totalRaised DESC
webhook_events: processedAt ASC, createdAt ASC (for unprocessed queue
polling)
```

4.5 Atomic Counter Pattern (Race-Condition Safe)

Never use read-modify-write for counters. All aggregate counter updates use Firestore atomic increments:

```
// In Cloud Function — CORRECT atomic increment
await admin.firestore().doc(`gaushalas/${gaushalId}`).update({
  totalRaised: admin.firestore.FieldValue.increment(amountPaise),
  totalDonors: admin.firestore.FieldValue.increment(isNewDonor ? 1 : 0),
});
```

For very high-frequency counters (festival campaigns with 100+ donations/minute), use **distributed counter sharding** (10 shards, sum on read). Sharding is activated automatically by `computeTransparencyScore` when a Gaushala's hourly donation rate exceeds 60/hour.

PART 5 — PAYMENT ARCHITECTURE

5.1 Razorpay Route Integration

Architecture Principle: Every payment-related Cloud Function is **idempotent**. Every webhook is logged first, processed second. No synchronous side effects in the webhook response path.

Step-by-Step Payment Flow

```
Step 1: Donor selects Gaushala + amount + category in Flutter app

Step 2: Flutter calls Cloud Function: POST /createOrder
  - Body: { gaushalId, amountPaise, category }
  - userId extracted from JWT only — never from request body
  - Cloud Function:
    a. Validates amountPaise against platform_config.maxDonationAmountSingle
    b. Checks donor's daily donation limit
(platform_config.maxDonationAmountDaily) via Upstash Redis counter
    c. Checks FCRA restriction: if Gaushala is not FCRA-registered, set
international=false on Razorpay order
    d. Fetches Gaushala's razorpayLinkedAccountId from Firestore
    e. Calculates platform fee from platform_config (server-side — never
trust client)
    f. Creates Razorpay Order with Route transfer rules:
       {
         amount: amountPaise,
         currency: "INR",
         transfers: [{
           account: gaushalLinkedAccountId,
           amount: amountPaise - platformFeePaise,
           currency: "INR",
           on_hold: 0
         }]
       }
    g. Creates Firestore donation doc with status=created, razorpayOrderId
    h. Returns { orderId, amount, currency, key } to Flutter

Step 3: Flutter opens Razorpay checkout with the returned orderId

Step 4: Razorpay processes payment (UPI / Card / Netbanking / Wallet)

Step 5: Razorpay sends webhook POST to Cloud Function endpoint (HTTPS)

Step 6: Webhook handler (FAST PATH — must complete in < 500ms):
  a. Extract Razorpay-Signature header
  b. Verify HMAC-SHA256 signature using Razorpay webhook secret from Secret
Manager
  c. Validate timestamp: reject if |now - webhook_timestamp| > 300 seconds
(replay attack prevention)
  d. Write to webhook_events with razorpayEventId as UNIQUE key
     → ON CONFLICT (razorpayEventId): do not process again — idempotency
  e. Return HTTP 200 immediately

Step 7: Async webhook processor (Cloud Function triggered by Firestore write
to webhook_events):
  - For payment.captured:
    a. Look up donationId by razorpayOrderId
    b. CRITICAL: Verify captured amount == original order amount in Firestore
(amount manipulation check)
    c. Update donation status=captured in Firestore
    d. Atomic increment Gaushala totalRaised, totalDonors
    e. Atomic increment user totalDonated, donationCount
    f. Write to Cloud SQL financial_transactions ledger (immutable)
```

```
    g. Publish Pub/Sub message: DONATION_CAPTURED { donationId } (triggers
FCM + PDF)
    h. Write to audit_log
    i. Mark webhook_event processedAt = now()

Step 8: FCM/SMS notification sent (Pub/Sub subscriber, separate Cloud
Function)
Step 9: PDF generation job triggered (Pub/Sub subscriber → Cloud Run PDF
service)
```

Failed Payment Flow

```
- Webhook: payment.failed → update donation status=failed
- SMS fallback sent immediately (rural donor may have no FCM data
connectivity)
- FCM also sent if device is reachable
- Donor retry: Flutter app shows "Retry Payment" button → new createOrder
call
  (new orderId; original donation doc updated with new orderId for audit
trail)
- Subscription payment failure: after 3 consecutive failures, subscription
status=payment_failed,
  Gaushala admin notified, donor notified via FCM + SMS with retry link
```

Refund Flow

```
- Super Admin initiates in admin panel: processRefund(donationId, reason,
reasonCategory)
  a. Caller verified as Super Admin via JWT custom claim (not header)
  b. Check donation status == captured (cannot refund already-refunded or
failed)
  c. Create refund_requests record status=approved (Super Admin bypass for
direct initiation)
     OR donor creates request → Super Admin approves → Cloud Function
processes
  d. Call Razorpay Refund API
  e. On webhook refund.processed:
     - Update donation status=refunded
     - Atomic decrement Gaushala totalRaised (do NOT decrement totalDonors —
audit integrity)
     - Insert reversal row in Cloud SQL financial_transactions (never delete
original)
     - Write to audit_log
     - Notify donor via FCM + SMS
```

5.2 Suspended Gaushala Donation Handling

Policy when Gaushala is suspended (status = 'suspended'):

1. Cloud Function `onGaushalaStatusChange` fires on Firestore write.
2. All active subscriptions for this Gaushala → status = 'paused'. Donors notified via FCM + SMS.
3. All pending payouts → status = 'on_hold'. Funds held in platform escrow.

4. Gaushala's listing removed from all search results and discovery feeds immediately.
5. Verified badge removed from profile (profile still accessible by direct URL with "Suspended" banner).
6. Super Admin has 30 days to resolve:
    - **Restore:** Gaushala provides remediation proof → Super Admin reactivates → subscriptions resume → held payouts released.
    - **Deactivate permanently:** All held payouts returned to donors as refunds → all active subscriptions cancelled → donors notified with full explanation.

5.3 UPI Direct Donation — REMOVED

UPI direct donation (showing UPI ID on profile for out-of-platform payments) is **removed from the platform**. All donations must flow through Razorpay Route. Platform supports UPI *through* Razorpay checkout as a payment method.

5.4 Minimum Payout Threshold & Payout Cycle

- `platform_config.minimumPayoutAmount` defaults to ₹100 (10000 paise).
- `platform_config.payoutCycleDays` defaults to 7 (weekly payouts on Fridays).
- Gaushalas below threshold accumulate until threshold is met; dashboard shows progress.
- Payout initiation is automated via Cloud Scheduler; Super Admin can manually trigger.
- Every payout generates a GST invoice (if platform fee > 0) stored in Firebase Storage.

PART 6 — CLOUD FUNCTIONS SPECIFICATION

All Cloud Functions written in TypeScript. All functions validate the caller's Firebase Auth JWT. All financial functions write to `audit_log`. All functions implement structured logging (JSON) to Cloud Logging.

Function	Trigger	Key Behaviors
`createRazorpayOrder`	HTTPS (authenticated)	Server-side fee calculation; FCRA check; rate limit via Upstash Redis; idempotency via orderId
`receiveWebhook`	HTTPS (public — signature verified)	Log only; return 200 immediately; triggers async processor
`processWebhookQueue`	Firestore trigger on `webhook_events`	Idempotent; amount validation; atomic increments; Pub/Sub publish
`generateTaxCertificate`	Pub/Sub (DONATION_CAPTURED) +	Async via Cloud Run PDF service; never synchronous

Function	Trigger	Key Behaviors
	HTTPS (manual)	
`initiateRefund`	HTTPS (Super Admin custom claim required)	Full refund lifecycle; audit log; donor notification
`createGaushalaLinkedAccount`	HTTPS (Super Admin only)	Razorpay Route KYC; store linked account ID
`onGaushalaStatusChange`	Firestore trigger on `gaushalas`	Suspend: pause subscriptions, hold payouts, notify donors
`scheduleRecurringPayments`	Cloud Scheduler (daily 9 AM IST)	Find subscriptions due today; create Razorpay charges; SMS fallback
`sendCampaignNotification`	HTTPS (Gaushala Admin — JWT-verified admin of that gaushalId)	FCM batch (sendMulticast, 500/batch); rate limit: 3/day per Gaushala
`cleanupExpiredOrders`	Cloud Scheduler (hourly)	Expire orders older than 30 min; update status=expired
`computeTransparencyScore`	Cloud Scheduler (daily 2 AM IST)	See Part 18 for scoring algorithm
`exportDonorReport`	HTTPS (Gaushala Admin)	Rate limited 5/day; watermarked; audit log; only consented donors
`deleteUserData`	HTTPS (authenticated — own account only)	DPDP Act compliance; PII deleted; financial records retained with legal_hold marker
`generateAnnual80gBundle`	Cloud Scheduler (April 1, 00:01 IST) + HTTPS (manual)	Batch-generate all Form 10BE PDFs for prior financial year; async
`onNewGaushalaApplication`	Firestore trigger on `gaushalas` status=pending	FCM + email to Super Admin; acknowledgement email to applicant
`rotatePanEncryptionKey`	Cloud Scheduler (quarterly)	Re-encrypt all PANs with new KMS key version; zero-downtime rotation
`healthCheck`	Cloud Scheduler (every 5 min)	Tests: Firestore read/write, Razorpay API ping, Cloud SQL connectivity; PagerDuty alert on failure
`processGaushalaKyc`	HTTPS (Super Admin only)	Approve/reject each KYC document; trigger linked account creation on full approval

Function	Trigger	Key Behaviors
`createPartitionForNextMonth`	Cloud Scheduler (1st of each month)	Auto-creates next month's Cloud SQL partition for financial_transactions
`monthlyReconciliation`	Cloud Scheduler (1st of each month, 3 AM IST)	Compare Razorpay payout reports vs Cloud SQL ledger; flag discrepancies to Super Admin
`processGaushalaKycExpiry`	Cloud Scheduler (daily)	Check 80G/12A/FCRA expiry dates; send 60-day and 30-day warnings; halt certificates on expiry

---

PART 7 — FLUTTER APP ARCHITECTURE

7.1 State Management — Riverpod

```
lib/
├── core/
│   ├── providers/          # Global providers (auth, config, connectivity)
│   ├── router/             # GoRouter with auth guards + deep link handlers
│   ├── theme/              # Design tokens, dark mode
│   ├── i18n/               # ARB files for en, hi (Phase 1); gu, mr, ta
(Phase 4)
│   └── utils/              # Formatters, validators, PII scrubber
├── features/
│   ├── auth/
│   │   ├── data/           # Firebase Auth repository
│   │   ├── domain/         # AuthState, User model
│   │   └── presentation/   # OTP screen, profile setup, account recovery
│   ├── discovery/
│   │   ├── data/           # Typesense repository, Firestore queries, Hive
cache
│   │   ├── domain/         # GaushalaListState, SearchFilters
│   │   └── presentation/   # Home, Search, Filters, Near Me map
│   ├── gaushala_profile/
│   │   ├── data/           # Firestore + Cloudflare Stream
│   │   ├── domain/         # GaushalaProfileState
│   │   └── presentation/   # Profile, Gallery, Updates feed, Campaigns
│   ├── donation/
│   │   ├── data/           # createOrder CF call, Razorpay checkout, outbox
│   │   ├── domain/         # DonationFlowState (multi-step FSM)
│   │   └── presentation/   # Amount selection, category, payment, success
animation
│   ├── history/
│   ├── subscriptions/
│   ├── certificates/       # 80G download, share, annual bundle
│   ├── notifications/
│   └── settings/           # Language, notifications, privacy, account
deletion
```

```
└── shared/
    ├── widgets/
    └── services/           # FCM, Analytics (PII-scrubbed), SecureStorage,
Connectivity
```

7.2 Offline Architecture (Bharat-First)

```
// Outbox pattern — two cases handled differently:
// Case A: Network dropped BEFORE createOrder was called
//   → Store DonationIntent in Hive outbox; retry createOrder when online
// Case B: Order created, network dropped BEFORE Razorpay checkout
//   → Store existingOrderId in Hive; on reconnect, check order status
//   → If order still 'created' (< 30 min old): re-open Razorpay checkout
//   → If order expired: create new order
// Case C: Payment completed — webhook handles server-side. App polls
donation status
//   on reconnect using donationId (no duplicate risk).

class DonationOutbox {
  final String tempId;         // Local UUID
  final String gaushalId;
  final int amountPaise;
  final String category;
  final String? existingOrderId; // null = order not yet created
  final DateTime orderCreatedAt; // To check 30-min expiry
  final int retryCount;
  final DateTime firstAttemptAt;
}
```

Cached data (Hive):

- Gaushala listings (last 50 viewed) — refreshed every 15 min when online; shown stale with "Updated X min ago"
- Donation history (last 100 records) — refreshed on app resume
- User profile + notification preferences
- Platform config (kill switches) — refreshed on app start

Connectivity rule: Payment initiation screens require active internet. Show "No connection — payment requires internet" if offline. Never attempt to create an order while offline (prevents partial state confusion).

7.3 Authentication Flow

```
1. Phone number entry → Firebase Auth sendOTP (branded sender: GauSevak)
2. OTP entry (60-second timeout; resend after 30s; max 5 OTP requests/hour
per phone)
3. OTP tested on: Jio, Airtel, Vi, BSNL — confirmed delivery < 10 seconds
4. New user: Profile setup (name required; email required — used for account
recovery + 80G)
5. Google Sign-In alternative (email mandatory, captured on first sign-in)
6. Token refresh: Firebase SDK handles silently; auth state listener rebuilds
Riverpod providers
```

```
7. Session revocation: Super Admin can call Firebase Admin
revokeRefreshTokens(uid) — user force-logged-out on next API call
8. Rooted/compromised device detection:
   - Android: Play Integrity API — MEETS_DEVICE_INTEGRITY check on app start
   - iOS: App Attest — attestation key verified on app start
   - Compromised device: payment flows blocked; other features allowed with
warning banner
9. Account recovery (phone change): support-assisted via email verification +
last-3-donations knowledge auth
   Automated self-service recovery via email OTP in Phase 3.
```

7.4 SSL Certificate Pinning — Rotation Strategy

```
// Two pins always active: current + next
// Pin format: SHA-256 of SubjectPublicKeyInfo (SPKI)

// In http_certificate_pinning configuration:
List<String> activePins = RemoteConfig.getString('ssl_pins').split(',');
// Firebase Remote Config pushes new pin 30 days before certificate
expiration
// Both old and new pins accepted during 30-day rotation window
// After rotation: old pin removed from Remote Config on certificate expiry
date

// CI test: automated pin rotation test runs weekly
// Verifies: (a) current pin matches live certificate, (b) backup pin is pre-
loaded
```

Pin rotation runbook (must be in operations wiki):

1. 30 days before cert expiry: generate new cert, extract SPKI hash, push to Remote Config as second pin.
2. 7 days before: push app release with updated pin list (if pins are bundled in app — prefer Remote Config).
3. On cert rotation day: remove old pin from Remote Config.
4. Monitor Sentry for certificate pinning errors for 48 hours post-rotation.

7.5 Security Hardening

- **Secure Storage:** `flutter_secure_storage` for auth tokens. Android Keystore + iOS Keychain.
- **Screenshot prevention** on payment screens and 80G certificate screens: `FLAG_SECURE` (Android) + `ignoresSiblingOrder` trick (iOS).
- **Obfuscated builds:** `flutter build apk --obfuscate --split-debug-info=./debug-info`
- **No secrets in app bundle:** Firebase config (google-services.json, GoogleService-Info.plist) contains only the public Firebase project config — treated as public. All secrets in Cloud Functions via Secret Manager.

- **Root detection:** Integrated with Play Integrity (Android) and App Attest (iOS). Blocks payment flows only — does not block the entire app (avoids false positives on legitimate custom ROMs used in India).
- **Jailbreak detection** (iOS): `flutter_jailbreak_detection` package as secondary check (App Attest is primary).

7.6 Deep Links

```
final router = GoRouter(
  routes: [
    GoRoute(
      path: '/g/:slug',
      builder: (_, state) => GaushalaProfileScreen(slug:
state.pathParameters['slug']!),
    ),
    GoRoute(
      path: '/c/:campaignId',
      builder: (_, state) => CampaignScreen(id:
state.pathParameters['campaignId']!),
    ),
    GoRoute(
      path: '/receipt/:donationId',
      redirect: (context, state) {
        final auth = ref.read(authProvider);
        if (!auth.isLoggedIn) return '/login?returnUrl=${state.uri}';
        return null;
      },
      builder: (_, state) => DonationReceiptScreen(donationId:
state.pathParameters['donationId']!),
    ),
  ],
);
```

PART 8 — ADMIN PANELS

8.1 Gaushala Admin Panel (React)

Tech stack: React 18 + Ant Design 5 + React Query + Axios (calling Cloud Function APIs)

Access control: Gaushala admin logs in via Firebase Auth (same phone/Google). JWT verified in Cloud Functions. React UI reads `gaushala_admins` to show which Gaushalas the user manages (multi-Gaushala support from day one).

Donor Data Export Policy:

- Rate limit: 5 exports/day per admin account (enforced in Cloud Function via Upstash Redis)
- Every export writes to `audit_log`
- Export CSV is watermarked: admin userId + timestamp embedded in file metadata

- Only donors who consented to "share my name with this Gaushala" are included
- Phone and email in export are hashed unless donor explicitly opted into communication
- Export format: pre-formatted for Form 10BD filing (Gaushala's tax obligation)

8.2 Super Admin Panel (React)

Access: Firebase custom claim `superAdmin: true` — set manually via Firebase Admin SDK. Not self-registerable. Google Sign-In with Google Workspace enforces organisational 2FA.

PAN Access Controls:

- PAN retrieval is a separate Cloud Function endpoint requiring a `reason` parameter
- Every PAN retrieval logged to `audit_log` with reason
- Automated alert: > 10 PANs accessed in 1 hour by one admin → PagerDuty P1 → session auto-revoked
- Bulk PAN export (> 100 records) requires two Super Admin approvals (4-eyes principle) — approval flow built into Super Admin panel

Financial Reconciliation View:

- Monthly comparison: Cloud SQL `monthly_reconciliation` view vs Razorpay Route settlement report
- Discrepancies highlighted in red; export to CSV for CA review
- TDS tracking: donations above ₹10,000 threshold flagged for Gaushala's TDS compliance

---

PART 9 — VIDEO STORAGE ARCHITECTURE

Cloudflare Stream for all Gaushala gallery videos:

- Gaushala admin requests pre-signed upload URL from Cloud Function
- Cloud Function calls Cloudflare Stream API → returns direct upload URL
- Flutter/React uploads directly to Cloudflare Stream (bypasses Firebase Storage for video)
- Cloudflare transcodes to adaptive bitrate HLS automatically (360p, 480p, 720p)
- Upload creates `updates/{updateId}` with status = `pending_moderation` and Cloudflare Stream video ID
- Super Admin moderation: approve → status = `approved`, video URL stored; reject → Cloudflare video deleted
- Flutter uses `video_player` package with Cloudflare HLS URL + thumbnail
- **Platform policy limits:** 500MB raw upload, 10-minute maximum duration
- **Cost model:** ~$5/1,000 min stored + $1/1,000 min delivered — no egress surprise at scale

---

PART 10 — NOTIFICATION ARCHITECTURE

Three-Layer Delivery System

Channel	When Used	Provider
FCM Push	App in background/closed; all standard notifications	Firebase Cloud Messaging
In-App Real-time	App is open; live campaign donation ticker	Firestore real-time listeners
SMS Fallback	All payment-critical notifications, regardless of FCM status	MSG91 / Kaleyra

SMS Fallback (non-optional for payment events)

Templates (DLT-registered — mandatory for Indian SMS delivery):

- Donation captured: `GauSevak: Rs.{amount} donated to {gaushala}. Receipt #{receipt}. Download: {link} -GauSevak`
- Subscription failed: `GauSevak: Monthly donation of Rs.{amount} to {gaushala} failed. Retry: {link} -GauSevak`
- Certificate ready: `GauSevak: Your 80G certificate FY {year} is ready. Download: {link} -GauSevak`
- Gaushala suspended: `GauSevak: {gaushala} is suspended. Your recurring donation is paused. Funds are safe. -GauSevak`

DLT Registration: All SMS templates must be registered on the TRAI DLT (Distributed Ledger Technology) platform before go-live. Registration via MSG91/Kaleyra takes 3–5 business days. Include in Stage 0 checklist.

FCM Batch Sending

- Campaign notifications use `sendMulticast` (500 tokens per batch) in a loop inside Cloud Function
- Rate limit: 3 campaign notifications per Gaushala per day (Upstash Redis counter)
- Donor notification preferences respected: if user opted out of `campaign_update` type, skip that userId
- Topic-based subscriptions for festival campaigns: donors can subscribe to `gopashtami_2025` topic

---

PART 11 — SCALABILITY & MIGRATION PLAN

Phase 1–2 (0–10,000 users) — Firebase Stack

- Firebase Cloud Functions + Firestore handles this comfortably
- Monitor: Cloud Function execution time, Firestore read/write costs daily
- Alerts: Function P99 > 2s, Firestore daily cost > ₹5,000, Cloud SQL CPU > 60%

Phase 3 (10,000–100,000 users) — Firebase + Cloud Run + Cloud SQL Replica

- PDF generation already on Cloud Run (no migration)
- Video already on Cloudflare Stream (no migration)
- Add Cloud SQL read replica (async replication) for analytics dashboard queries
- Enable Firestore distributed counters for Gaushalas with > 60 donations/hour (auto-detected by computeTransparencyScore)
- Typesense: upgrade from single-node to 3-node HA cluster on Cloud Run
- Move Gaushala gallery images behind Cloudflare CDN (Cloudflare proxied Firebase Storage URL)
- Cache invalidation: Cloud Function calls Cloudflare Cache Purge API on Gaushala profile updates

Phase 4 (100,000–1,000,000 users) — Cloud Run Microservices Migration

Trigger: Firebase Cloud Functions bill > ₹2L/month OR cold start P95 > 3 seconds

6-month migration path (plan at 75K users, execute at 150K):

Component	Migration
Primary Database	Cloud SQL PostgreSQL 15 Multi-AZ (asia-south1) becomes primary; Firestore retained for real-time features only
Backend	Node.js microservices on Cloud Run (stay GCP for Firebase Auth integration)
Auth	Firebase Auth stays — cheap, India OTP coverage
Queue	Cloud Pub/Sub for all async processing (already partially in place)
Cache	Upstash Redis already in place — expand TTL and key namespace for query cache
Search	Typesense cluster already in place — scale up nodes
CDN	Cloudflare already in place — no change
Analytics	BigQuery for post-100K analytical queries (export Cloud SQL → BigQuery nightly)

Zero-downtime migration strategy: Dual-write (both Firestore and Cloud SQL) for 4 weeks → validate Cloud SQL completeness → switch reads to Cloud SQL → disable Firestore writes for migrated collections.

PART 12 — DEVOPS & CI/CD

12.1 Repository Structure (Monorepo)

```
gau-sevak/
├── apps/
│   ├── mobile/              # Flutter app
│   ├── admin-gaushala/      # React Gaushala admin
│   └── admin-super/         # React Super Admin
├── backend/
│   ├── functions/           # Firebase Cloud Functions (TypeScript)
│   │   ├── src/
│   │   │   ├── payment/     # createOrder, webhook, refund
│   │   │   ├── certificates/ # 80G generation
│   │   │   ├── admin/       # gaushala KYC, payout management
│   │   │   ├── notifications/ # FCM, SMS dispatch
│   │   │   └── scheduled/   # Cron jobs
│   │   └── tests/           # Unit + integration tests
│   └── pdf-service/         # Cloud Run Puppeteer service
│       ├── templates/        # Handlebars Form 10BE template (CA-approved)
│       └── Dockerfile
├── infra/
│   ├── terraform/           # GCP infrastructure as code (Cloud SQL, Cloud
Run, IAM)
│   └── firestore/           # Security rules, indexes, emulator test data
├── docs/
│   ├── adr/                 # Architecture Decision Records
│   └── runbooks/            # Incident response playbooks
└── .github/
    └── workflows/           # CI/CD pipelines
```

12.2 CI/CD Pipeline

```
# PR: lint → unit tests → Firestore emulator integration tests → security
rules tests
# Merge to main: deploy to staging (auto)
# Tag release vX.Y.Z: deploy to production (manual approval gate required)

Environments:
  development:  Firebase project: gau-sevak-dev   (engineers use freely)
  staging:      Firebase project: gau-sevak-stg   (mirrors production config)
  production:   Firebase project: gau-sevak-prod  (IAM-locked; only CI/CD
service account writes)
```

Environment secrets isolation:

- Production Razorpay key: only in GitHub Actions environment `production` (requires reviewer approval)
- Staging uses Razorpay test mode keys exclusively
- Developers never have access to production Firebase project directly — all changes via CI/CD

12.3 Database Migration Strategy

- Cloud SQL migrations managed with **Flyway** (versioned SQL files, repeatable migrations)
- All migrations are backward-compatible (new columns nullable with defaults; old columns deprecated before removal)
- Staging migration runs 24 hours before production; any migration failure stops production deployment
- Down migrations (rollback scripts) written alongside every up migration — enforced in PR checklist

12.4 Mobile Release (Codemagic)

- Android: staged rollout 5% → 25% → 100% (Play Store internal → production track)
- iOS: TestFlight internal testing → external testing → App Store submission
- Firebase Remote Config controls feature rollout independently of app versions — feature flags can enable/disable features without a new app release
- App version pinning: if a critical security bug is found, Remote Config can set `min_supported_version` to force-update older clients

12.5 Feature Flags (Firebase Remote Config — Emergency Kill Switches)

```
payment_enabled              # Master kill switch for all donations
recurring_donations_enabled
video_upload_enabled
near_me_search_enabled
campaign_notifications_enabled
maintenance_mode             # Shows maintenance screen in Flutter app
ssl_pins                     # Comma-separated current+next SSL pin hashes
min_supported_version        # Force-update below this version
pdf_service_url              # Cloud Run PDF service URL (updateable without
redeploy)
```

12.6 Disaster Recovery

Metric	Target
RTO (Recovery Time Objective)	2 hours
RPO (Recovery Point Objective)	1 hour

Backup strategy:

- Firestore: automated daily export to Cloud Storage (Cloud Scheduler) + point-in-time recovery enabled on the project
- Cloud SQL: continuous WAL archiving to Cloud Storage using `pgBackRest` — achieves < 1-hour RPO; daily full backup retained for 30 days
- Firebase Storage: cross-region replication (asia-south1 primary; asia-east1 backup) enabled for certificates and KYC documents

- Secret Manager: GCP-managed geo-replication — no action needed

Quarterly DR drill:

- Restore Cloud SQL to point-in-time 6 hours ago in staging environment
- Verify financial_transactions row count matches expected
- Verify Firestore export can be imported to staging project
- Document actual RTO achieved; update runbook if > 2 hours

PART 13 — MONITORING & ALERTING

Business Metric Alerts (PagerDuty — P1/P2)

Alert	Threshold	Priority
Payment failure rate	> 5% in 5-min window	P1
Webhook processing lag	> 60 seconds	P1
Super Admin PAN bulk access	> 10 PANs in 1 hour	P1 (security)
Cloud Function P99 latency	> 3 seconds	P2
80G certificate failure rate	> 1%	P2
Cloud SQL connection pool utilization	> 80%	P2
Daily donation volume drop	> 50% vs 7-day average	P2 (silent failure detection)
Gaushala KYC document expiry	60 days before expiry	P3 (email only)

Technical Alerts (Firebase Alerts + Sentry)

- Crashlytics: crash-free session rate < 99.5% → Sentry alert
- Sentry: any new Cloud Function error type → immediate Slack notification
- Cloud SQL: CPU > 80%, storage > 80% → alert
- Cloudflare Stream: error rate > 1% → alert
- Typesense: query latency P99 > 500ms → alert

SLO Definitions (Service Level Objectives)

API / Service	Availability	Latency P99
createRazorpayOrder	99.9%	< 2 seconds
receiveWebhook (log + 200)	99.95%	< 500ms
processWebhookQueue (donation captured)	99.9% within 60 seconds	—
80G certificate delivery	99% within 5 minutes of capture	—
Gaushala search (Typesense)	99.5%	< 300ms
Flutter app (Firebase Hosting)	99.9%	—

PART 14 — COMPLIANCE AUTOMATION

Financial Year Automation

- **March 31, 11:59 PM IST:** `generateAnnual80gBundle` cron locks FY, begins batch PDF generation for all eligible donor × Gaushala combinations (24-hour processing window allowed)
- **April 1:** All FY certificates available for download in donor app and Gaushala admin export
- **May 31:** Automated reminder to Gaushala admins: file Form 10BD with IT Department using platform export
- **PMLA 5-year retention:** Records older than 5 years flagged read-only via Cloud SQL trigger; physical deletion blocked at DB level

KYC Expiry Monitoring

- `processGaushalaKycExpiry` runs daily, checks `gaushala_kyc_documents.expiryDate`
- **60 days before expiry:** Email to Gaushala admin + in-panel banner: "Your 80G certificate expires on {date}. Upload renewal to avoid interruption."
- **30 days before expiry:** App profile badge: "80G renewal pending" visible to donors
- **On expiry:** `eligible80g = false` for all new donations to this Gaushala; 80G certificate generation halted; existing certificates remain valid
- **FCRA certificate expiry:** Same 60/30-day flow; on expiry, international payments blocked immediately

80G Issuance Double-Claiming Prevention

- Each donation has a `certificateId` field. One donation → one certificate entry. Donor can download multiple times but cannot receive duplicate certificate numbers.
- If a donor also donates directly to the Gaushala outside the platform (UPI direct — which the platform has removed), the platform cannot prevent double 80G issuance by the Gaushala. The platform's ToS states it only issues certificates for platform-routed donations. This is documented and legally reviewed.

PART 15 — PHASE-WISE DELIVERY (REVISED)

Stage 0: Pre-Development (6 weeks — NON-NEGOTIABLE)

- [ ] Fintech lawyer confirms PA license structure under Razorpay marketplace
- [ ] CA reviews Form 10BE format; approves Handlebars certificate template

- [ ] ToS + Privacy Policy drafted, reviewed, published (DPDP Act 2023 compliant)
- [ ] FCRA legal structure confirmed; hard-block mechanism approved by lawyer
- [ ] ADRs written: Riverpod, React admin, async webhook, Cloud SQL dual-store
- [ ] Cloud SQL schema migrations written with all indexes (before any feature code)
- [ ] Firestore security rules written + automated tests via Firebase Emulator
- [ ] Razorpay Route merchant agreement reviewed; marketplace charitable use case confirmed with Razorpay account manager
- [ ] DLT SMS template registration initiated (3–5 business day lead time)
- [ ] GCP project setup: dev, staging, prod; Secret Manager; IAM roles locked down; Terraform applied
- [ ] Play Store and App Store developer accounts created

Phase 1 — MVP (12 weeks)

Single Gaushala, real donations, production-quality foundations:

- Firebase Auth (phone OTP + Google) with mandatory email capture
- Single Gaushala profile
- Full donation flow: Razorpay Standard → async webhook queue → Firestore → Cloud SQL ledger
- 80G certificate generation (Form 10BE compliant, async Cloud Run PDF)
- Donation history with filter/pagination
- FCM push + SMS fallback for payment confirmation (DLT registered)
- Super Admin panel: manual Gaushala content management, donation view, refund initiation
- Firestore security rules fully tested (automated RLS test suite in CI)
- Android only; 500 beta users
- Load test: 500 concurrent donation attempts

Phase 2 — Marketplace (14 weeks)

- Multi-tenant: full `gaushalas` collection, `gaushala_admins` junction table
- Gaushala KYC onboarding + Razorpay Route Linked Account creation
- Gaushala discovery: Typesense search + Firestore GeoQuery (near me)
- Gaushala Admin panel (React): content management, anonymized donor view, financial reports
- Razorpay Route payout management with GST invoice generation
- iOS app released (App Attest integration)
- FCRA blocking enforced at createOrder
- Suspended Gaushala donation handling
- Load test: 5,000 concurrent users
- External security review focused on payment flows, RLS, IDOR

Phase 3 — Scale (14 weeks)

- Recurring donations (Razorpay Subscriptions + Route — 3 weeks testing budget)

- Campaign fundraising with approval workflow
- WhatsApp sharing (deep links)
- Full analytics dashboard (Cloud SQL-backed, materialized views)
- Offline support (Hive outbox pattern)
- Festival surge preparation: distributed counters, load test 50,000 concurrent
- Penetration test by external firm (budget ₹3–8L)
- Cloud Run PDF service performance tuning
- Play Integrity API + App Attest enforcement on payment flows
- 50,000 users, 100+ Gaushalas

Phase 4 — Growth (Ongoing)

- Corporate CSR portal (B2B, white-label React app)
- FCRA-compliant foreign donation pathway (FCRA-registered Gaushalas only)
- Vernacular languages: Gujarati, Marathi, Tamil (i18n ARB files)
- Cow sponsorship / adoption programs
- Tally / Zoho Books API integration for Gaushala financial accounting
- Architecture migration assessment at 75K users; execute Cloud Run migration at 150K users
- BigQuery analytics pipeline for > 100K user scale

PART 16 — COST MODEL

Monthly Infrastructure Estimates

Scale	Firebase	Cloud SQL	Cloudflare Stream	Typesense	MSG91 SMS	Upstash Redis	Total Est.
500 users (MVP)	₹2,100	₹1,500	₹500	₹0 (free)	₹500	₹0 (free)	~₹5,000/mo
5K users	₹4,000	₹3,000	₹2,000	₹1,500	₹2,000	₹500	~₹13,000/mo
50K users	₹15,000	₹8,000	₹10,000	₹5,000	₹15,000	₹2,000	~₹55,000/mo
100K users	₹35,000	₹20,000	₹25,000	₹10,000	₹40,000	₹5,000	~₹1,35,000/mo

Razorpay fees on ₹10 Cr annual volume: ₹15–25L/year depending on payment method mix (UPI is cheapest; card/netbanking is highest).

One-time pre-launch costs:

- Legal (PA, ToS, FCRA, PMLA policy): ₹5–15L
- CA (80G Form 10BE compliance + annual support): ₹1–3L/year
- Penetration test: ₹3–8L

- Load testing infrastructure: ₹50K–2L
- Apple Developer account: ₹7,500/year
- DLT SMS registration: ₹0–5K (via MSG91/Kaleyra)

PART 17 — WHAT MUST NEVER BE DONE

These are hard constraints. No exception, no workaround, no "we'll fix it later":

1. **Never process a Razorpay webhook synchronously.** Log → return 200 → process async. Always.
2. **Never read-modify-write Gaushala counters.** Atomic increments only. Always.
3. **Never generate 80G PDFs inside a webhook handler.** Async Cloud Run job. Always.
4. **Never take userId from the request body for authorization.** Always extract from JWT. Always.
5. **Never store bank account details or PAN in the main `gaushalas` or `users` documents.** Separate encrypted subcollection. Always.
6. **Never allow a Gaushala's international payment without checking FCRA status.** Hard block in createOrder. Always.
7. **Never delete from `audit_log` or `financial_transactions`.** Append-only forever.
8. **Never send raw PAN numbers to analytics, logs, or third-party tools.** Encrypted at rest; only last-four shown in UI.
9. **Never allow Super Admin to access PAN data without writing to `audit_log`.** Every access audited.
10. **Never record a donation as captured without verifying the captured amount matches the order amount.** Non-negotiable.
11. **Never use Algolia for indexing donor data or donation history.** Only Gaushala public profiles are indexed.
12. **Never send more than 500 FCM tokens in a single `sendMulticast` call.** Batch in loops.
13. **Never expose the Razorpay webhook secret or Cloud KMS key in any log, error message, or API response.** All secrets via Secret Manager only.
14. **Never allow a Gaushala admin to export donor PII without an audit_log entry and rate-limit check.** Both enforced server-side, not client-side.
15. **Never deploy a schema migration without a corresponding rollback (down) migration.**

PART 18 — TRANSPARENCY SCORE ALGORITHM

Purpose

The transparency score (0–100) is a computed field on each Gaushala that represents how trustworthy and active they are, based on verifiable on-platform behaviour. It is used in search ranking and displayed as a badge to donors. It is **not** self-reported.

Scoring Components (Computed daily by `computeTransparencyScore`)

Component	Max Points	Logic
Profile completeness	10	+2 each: description, founding year, GPS location, contact phone, cover photo
KYC verification	20	+20 if all required KYC docs are approved and not expired
Update frequency	25	Sliding 90-day window. ≥ 12 updates = 25 pts; 8–11 = 20; 4–7 = 12; 1–3 = 5; 0 = 0
Cow count staleness	15	Updated within 7 days = 15; within 30 days = 10; within 90 days = 5; older = 0
Donor response rate	15	% of donor thank-you messages replied to (via platform) in 7 days. ≥ 80% = 15; 50–79% = 8; < 50% = 0
Payout health	10	No failed/disputed payouts in last 90 days = 10; any disputed = 0
No active complaints	5	No open Super Admin complaints in last 30 days = 5; any open = 0

Fraud-Resistance Rules

- Cow count is only accepted from the Gaushala admin (not self-reported via donor). Super Admin can flag a count as "unverified" which reduces the cow count staleness score to 0 until verified.
- Update frequency counts only **approved** updates (passed content moderation), not pending/rejected ones.
- A Gaushala cannot game the score by submitting updates that are immediately rejected for low-quality content — rejected updates do not count.
- Score is recalculated by a Cloud Function with Admin SDK access, never writable by clients.

PART 19 — TESTING STRATEGY

Unit Tests (Cloud Functions)

- Every Cloud Function has unit tests covering: happy path, edge cases, error scenarios
- Mock Firebase Admin SDK and Razorpay API client in tests
- Coverage target: 80% line coverage minimum; 100% coverage for payment-critical functions
- Test runner: Jest (TypeScript)

Firestore Security Rules Tests

- All security rules tested using Firebase Emulator Suite + `@firebase/rules-unit-testing`
- Test scenarios per collection:
  - Unauthenticated access → expect deny
  - Donor reading own data → expect allow
  - Donor reading another donor's data → expect deny
  - Gaushala admin reading their own Gaushala → expect allow
  - Gaushala admin reading another Gaushala → expect deny
  - Gaushala admin writing financial data → expect deny
  - Super Admin operations → expect allow
- These tests run in CI on every PR. PR cannot merge if rules tests fail.

Integration Tests

- Payment flow: createOrder → mock webhook → verify Firestore donation status + Cloud SQL ledger row + FCM call
- Refund flow: initiate refund → mock Razorpay refund webhook → verify status + reversal row in Cloud SQL
- 80G generation: captured donation with PAN → verify PDF job queued → verify certificate doc created

Load Tests (k6)

Phase	Scenario	Target
Phase 1	Simultaneous createOrder calls	500 concurrent, < 2s P99
Phase 2	Gaushala search + profile load	2,000 concurrent, < 1s P99
Phase 3	Festival surge simulation	10,000 concurrent donations over 1 hour; 0 data corruption
Phase 3	Concurrent counter updates	1,000 simultaneous donations to same Gaushala; verify totalRaised accuracy

End-to-End Tests (Flutter integration tests)

- Full donation flow: login → discover Gaushala → select amount → mock Razorpay checkout → success screen → receipt visible in history
- 80G flow: PAN entry → donation capture → certificate available in certificates tab
- Offline flow: disconnect network → browse cached Gaushalas → re-connect → outbox processes

Security Tests

- IDOR test: authenticated as donor A, attempt to fetch donor B's donation → expect 403
- Amount manipulation: create order for ₹10, submit webhook with ₹1 captured → expect donation marked as failed (amount mismatch)
- Webhook replay: submit same Razorpay event ID twice → expect second to be silently ignored (idempotency)
- JWT spoofing: call processRefund with a user JWT (not Super Admin) → expect 403
- Rate limit: exceed 10 createOrder calls/minute from same IP → expect 429

PART 20 — LOCALIZATION

Phase 1: Hindi + English

- All UI strings in ARB files: `lib/core/i18n/intl_en.arb` and `intl_hi.arb`
- Language selection persisted in user settings (Hive)
- Default language: detected from device locale; fallback to Hindi if device is set to any Indian language, English otherwise
- Number formatting: `₹1,00,000` (Indian number system, not `₹100,000`)
- Date formatting: `DD/MM/YYYY` throughout
- All SMS templates registered in both Hindi and English on DLT

Phase 4: Gujarati, Marathi, Tamil

- ARB files added per language: `intl_gu.arb`, `intl_mr.arb`, `intl_ta.arb`
- Script support verified: Gujarati (gu), Devanagari (mr), Tamil (ta) fonts bundled in app
- Typesense Gaushala search configured with multi-locale tokenization

---

PART 21 — MONETIZATION CONFIGURATION

Monetization model is controlled entirely via `platform_config.platformFeePercent`. No code changes are needed to switch models.

Model	platformFeePercent	Notes
Zero commission	0	All donation reaches Gaushala
Operational fee	1.0–2.0	Covers infra + ops costs
Premium listings	0	Separate `featured` flag on Gaushala; flat ₹999/month via Razorpay recurring
Corporate CSR	Custom	Phase 4; separate pricing contract

Fee calculation (server-side in `createRazorpayOrder`, always):

```
const platformFeePaise = Math.floor(amountPaise * (config.platformFeePercent
/ 100));
const gstOnFeePaise = Math.floor(platformFeePaise * 0.18);
const netToGaushala = amountPaise - platformFeePaise; // GST on fee is
additional platform cost
```

GST handling: Platform absorbs GST on its fee (does not add it to the donor's amount). This is to maintain simple donor UX — "₹501 donated, ₹501 reaches [Gaushala] minus platform's %" rather than confusing donors with tax additions.

PART 22 — OPERATIONS RUNBOOK (FESTIVAL SURGE)

Pre-Surge Preparation (T-7 days before major festival: Gopashtami, Gau Puja, Diwali)

1. Load test at 3× expected peak traffic against staging environment (k6 scripts in `/docs/load-tests/`)
2. Pre-scale Cloud SQL instance compute tier (manually upgrade one tier in advance)
3. Enable distributed counters for top 20 Gaushalas by recent donation volume
4. Warm Cloud Functions by enabling minimum instance count (Firebase Cloud Functions min instances = 5 for createRazorpayOrder and receiveWebhook)
5. Notify Razorpay account manager of expected volume spike (they can pre-configure capacity)
6. Verify Cloudflare CDN cache hit rate > 80% for Gaushala profile pages
7. Ensure on-call engineer assigned with PagerDuty rotation active

During Surge (war room)

- Monitor PagerDuty and Sentry dashboards continuously
- If payment failure rate > 10%: check Razorpay status page first, then Cloud Function logs
- If Cloud SQL connections > 90%: enable read replica query routing immediately (manual switch in admin panel)
- If Firebase quota warnings appear: activate `near_me_search_enabled = false` (reduces Firestore geo queries) and `video_upload_enabled = false` to shed load

Post-Surge

- Run `monthlyReconciliation` function manually to verify no payout discrepancies
- Review Sentry for any new error patterns that appeared under load
- Archive load test results with peak metrics for future capacity planning

---

PART 23 — ARCHITECTURE DECISION RECORDS (ADR TEMPLATE)

All significant architecture decisions must be recorded as ADRs in `/docs/adr/`. Format:

```
# ADR-{number}: {Title}
Date: {YYYY-MM-DD}
Status: Accepted | Deprecated | Superseded by ADR-{n}

## Context
What situation required a decision?

## Decision
What was decided?

## Consequences
Positive: what does this enable?
Negative: what does this cost or constrain?

## Alternatives Considered
What was rejected and why?
```

Required ADRs (must exist before Phase 1 code starts):

ADR	Decision
ADR-001	Riverpod over BLoC for Flutter state management
ADR-002	React over Flutter Web for admin panels
ADR-003	Async webhook queue pattern (log-first, process-async)
ADR-004	Cloud SQL as immutable financial ledger alongside Firestore
ADR-005	Cloudflare Stream for video (vs Firebase Storage)
ADR-006	Typesense over Algolia for search
ADR-007	Upstash Redis for rate limiting (vs self-managed Redis)
ADR-008	Firebase Auth retained through all migration phases
ADR-009	Platform absorbs GST on fee (not passed to donor)
ADR-010	UPI direct donation removed from platform

This document is the single source of truth for the Gau-Sevak platform. Every implementation decision must be traceable to a requirement in this document. Any deviation requires an ADR with explicit justification. Parts 1–23 are complete and self-contained. No further clarification is required before beginning implementation.

GAU-SEVAK — MASTER SYSTEM PROMPT (COMPLETION)

Parts 24–40: Continuation from Part 23

This document is the direct continuation of the Gau-Sevak Master System Prompt (Version 2.0). Append everything below to the end of Part 23 of the main prompt. Do not alter any existing parts. Begin appending immediately after the ADR table.

PART 24 — GAUSHALA ONBOARDING & KYC WORKFLOW

24.1 Onboarding Lifecycle

A Gaushala transitions through these states in the `gaushalas/{gaushalId}` document:

```
DRAFT → SUBMITTED → UNDER_REVIEW → KYC_PENDING → KYC_APPROVED → ACTIVE
                                  ↘ REJECTED
                                  ↘ KYC_REJECTED → (can resubmit)
```

State definitions:

State	Meaning
`draft`	Gaushala started application; not yet submitted
`submitted`	All required fields filled; awaiting Super Admin review
`under_review`	Super Admin has opened the application
`kyc_pending`	Profile approved; KYC documents uploaded; awaiting document verification
`kyc_approved`	All documents verified; Razorpay Linked Account being created
`active`	Fully onboarded; accepting donations
`rejected`	Application rejected at profile stage (with reason)
`kyc_rejected`	Documents failed verification (with per-document reason)
`suspended`	Active → suspended by Super Admin (see Part 5.2)
`deactivated`	Permanently removed

24.2 Step-by-Step Onboarding Flow

Step 1 — Self-Registration (Gaushala Admin via web form)

The Gaushala Admin visits `gauSevak.in/apply`. Firebase Auth (phone OTP) creates their user account. They fill in:

- Organization name, type (Trust / Section 8 / Society / Informal)
- City, state, pincode, GPS coordinates
- Contact phone, contact email
- Brief description (minimum 100 characters)
- Founding year
- Approximate cow count

On submit: Firestore document created with `status = submitted`. Cloud Function `onNewGaushalaApplication` fires → notifies Super Admin via FCM + email.

Step 2 — Super Admin Profile Review

Super Admin reviews profile completeness and legitimacy in the Super Admin panel. If rejected at this stage, a `rejectionReason` is stored and the Gaushala Admin is notified via email + SMS with specific reason.

If approved: `status = kyc_pending`. Email to Gaushala Admin: "Your profile is approved. Please upload the following documents within 14 days."

Step 3 — Document Upload

Gaushala Admin uploads documents via the admin panel (React file upload → Firebase Storage signed URL → `gaushala_kyc_documents` collection):

Document	Required	Notes
Trust / Section 8 registration certificate	Yes	Must match entity name
12A registration certificate	Yes	Income tax exemption
80G registration certificate	Yes	With validity dates
Bank account proof (cancelled cheque)	Yes	Account name must match entity
PAN card of entity	Yes	Encrypted via KMS
FCRA certificate	Only if accepting foreign donations	
GST certificate	Only if applicable	
Representative ID proof	Yes	Director/trustee who signed application

Each upload:

1. Cloud Function generates Firebase Storage upload URL (5-minute TTL)

2. File uploaded directly from browser (bypasses Cloud Function for large files)
3. Firestore `gaushala_kyc_documents` record created with `status = pending`
4. Super Admin panel shows pending documents queue

Step 4 — Document Verification (Super Admin)

Per-document approval flow:

```
// Cloud Function: processGaushalaKyc
// Per-document decision:
await db.collection('gaushala_kyc_documents').doc(docId).update({
  status: decision, // 'approved' | 'rejected'
  reviewerNotes: notes,
  reviewedBy: superAdminUid,
  reviewedAt: admin.firestore.FieldValue.serverTimestamp(),
  expiryDate: extractedExpiryDate // Super Admin manually enters from
document
});
// Write to audit_log
```

If any document is rejected: `status = kyc_rejected`. Email + SMS to Gaushala Admin with per-document rejection reasons. They may re-upload rejected documents only.

Step 5 — Razorpay Linked Account Creation (Automated on Full KYC Approval)

When ALL required documents are `approved`, Cloud Function `createGaushalaLinkedAccount` fires:

```
// Razorpay Route — Create Linked Account
const linkedAccount = await razorpay.accounts.create({
  email: gaushala.email,
  profile: {
    category: 'not_for_profit',
    subcategory: 'charity',
    addresses: {
      registered: {
        street1: gaushala.address,
        city: gaushala.city,
        state: gaushala.state,
        postal_code: gaushala.pincode,
        country: 'IN'
      }
    }
  },
  legal_info: {
    pan: decryptPan(gaushala_sensitive.pan), // Only used here; never logged
    gst: gaushala.gstNumber ?? undefined
  }
});

// Store linked account ID (not PAN — PAN already in gaushala_sensitive)
await db.doc(`gaushalas/${gaushalId}`).update({
  razorpayLinkedAccountId: linkedAccount.id,
```

```
  bankAccountVerified: false, // True only after Razorpay verifies bank
account
  status: 'active'
});
```

Step 6 — Bank Account Linking (Gaushala Admin in Admin Panel)

Gaushala Admin enters bank details in the admin panel. Cloud Function encrypts via KMS and stores in `gaushala_sensitive`. A Razorpay stakeholder + bank account creation API call is made. Razorpay sends a ₹1 penny-drop verification. On Razorpay webhook `account.bank_account.added`: `bankAccountVerified = true`.

Only after `bankAccountVerified = true` can payouts be initiated.

24.3 Re-onboarding After Suspension

If a Gaushala is reactivated after suspension, a fresh KYC document review is mandatory before any new payouts are released. Existing approved documents are re-verified for expiry. The `status` transition from `suspended → active` requires explicit Super Admin action and writes to `audit_log`.

24.4 Gaushala Slug Assignment

Slugs are system-assigned from the Gaushala name, deduplicated:

```
function generateSlug(name: string): string {
  return name.toLowerCase()
    .replace(/[^a-z0-9\u0900-\u097F\s-]/g, '') // Allow Devanagari
    .replace(/\s+/g, '-')
    .substring(0, 50);
}
// On conflict: append -2, -3, etc.
// Slug is immutable once set (changing it breaks all shared links)
```

PART 25 — RECURRING DONATIONS ARCHITECTURE

25.1 The Complexity Problem

Razorpay Subscriptions + Razorpay Route is the most technically complex integration in this platform. The mandate (auto-debit permission) is created on the **platform's Razorpay account**, but the money must route to a **specific Gaushala's Linked Account** on each charge cycle. This requires careful sequencing:

```
Subscription mandate → Platform account
Each charge → Platform account receives full amount
```

```
→ Razorpay Route Transfer fires IMMEDIATELY after capture
→ Transfer: (amount - platformFee) → Gaushala Linked Account
```

This is the **"transfer after capture"** pattern, not the "transfer on order" pattern used for one-time donations. These are different Razorpay APIs.

25.2 Subscription Creation Flow

```
Step 1: Donor selects Gaushala + amount + frequency
(monthly/quarterly/yearly)

Step 2: Flutter calls Cloud Function: POST /createSubscription
  - Validates donor KYC level for amount (>₹50,000/year triggers enhanced
KYC)
  - Fetches Gaushala's razorpayLinkedAccountId
  - Creates Razorpay Subscription:
    {
      plan_id: <platform plan for this frequency>,
      quantity: 1,
      total_count: 120, // 10 years; effectively indefinite
      addons: [],
      notes: {
        gaushalId: gaushalId,
        donorUserId: userId,
        platformVersion: '2.0'
      }
    }
  - Creates Firestore subscriptions/{subscriptionId} with status='created'
  - Returns { subscriptionId, razorpaySubscriptionId, shortUrl }

Step 3: Flutter opens Razorpay Subscription checkout (shortUrl) for mandate
registration
  - UPI Autopay or NACH mandate
  - Donor authenticates mandate

Step 4: Razorpay webhook: subscription.activated
  → Update subscriptions/{id} status='active'
  → Calculate nextPaymentDate from subscription start + frequency
  → FCM + SMS to donor: "Monthly donation of ₹{amount} to {gaushala}
activated"

Step 5: Each charge cycle — Razorpay webhook: subscription.charged
  → Cloud Function receives webhook
  → Logs to webhook_events (idempotency)
  → Async processor:
    a. Creates donation/{donationId} with isRecurring=true, subscriptionId
    b. Calls Razorpay Transfer API: transfer (amount - fee) to Gaushala
Linked Account
    c. Logs transfer in payouts collection
    d. Updates subscriptions.nextPaymentDate
    e. Atomic increment on Gaushala + user counters
    f. Writes to Cloud SQL financial_transactions
    g. FCM + SMS receipt to donor
    h. Triggers 80G PDF generation if PAN on file
```

25.3 Razorpay Plan Management

Platform pre-creates Razorpay Plans for each frequency × amount combination. Plans are immutable (Razorpay does not allow plan amount changes). When a donor changes their subscription amount:

```
// NOT: edit existing subscription — Razorpay does not allow amount changes
// CORRECT FLOW:
// 1. Cancel existing Razorpay subscription (webhook: subscription.cancelled
fires)
// 2. Create new Razorpay subscription with new amount
// 3. Update Firestore subscriptions record with new razorpaySubscriptionId
// 4. Donor re-registers mandate for new subscription
// 5. Audit log: SUBSCRIPTION_AMOUNT_CHANGED with old and new amounts
```

Pre-created plans stored in `platform_config/subscription_plans`:

```
{
  monthly_100: { razorpayPlanId: 'plan_xxx', amount: 10000, interval:
'monthly' },
  monthly_251: { razorpayPlanId: 'plan_yyy', amount: 25100, interval:
'monthly' },
  ...
  custom: null // Custom amounts require on-demand plan creation
}
```

For custom amounts: Cloud Function creates a new Razorpay Plan on-demand. Plans are cached in `platform_config` to avoid duplicates.

25.4 Failed Payment Handling

```
Razorpay webhook: subscription.pending (payment due but not yet charged)
→ SMS reminder to donor 3 days before due date

Razorpay webhook: subscription.halted (payment failed after Razorpay retries)
→ Update subscriptions/{id}: status='payment_failed', failedPaymentCount++,
lastFailedAt
→ FCM + SMS to donor: "Donation failed. Tap to retry."
→ FCM to Gaushala Admin: "Subscription from [donor first name] has failed."
→ Donor retry flow: new one-time donation (not subscription resume) for
missed cycle
→ After 3 consecutive halted events: status='cancelled', donor notified
→ Gaushala aggregate: do NOT decrement counters — subscription was active;
failure is new state

Donor-initiated pause (max 3 months):
→ Flutter calls Cloud Function: pauseSubscription(subscriptionId)
→ Razorpay Subscription pause API called
→ subscriptions.status = 'paused', subscriptions.pausedAt = now()
→ FCM confirmation to donor
→ Cloud Scheduler daily check: if pausedAt > 90 days → auto-resume
(configurable in platform_config)
```

25.5 Subscription Ledger Integrity

Every subscription charge generates:

1. A `donations` document (full donation record)
2. A `financial_transactions` row in Cloud SQL (immutable ledger)
3. A Razorpay Transfer record (routed to Gaushala)

These three records are linked by `razorpayOrderId` / `subscriptionId`. Monthly reconciliation checks that every Cloud SQL row has a corresponding Razorpay Transfer. Discrepancies (charge received, transfer not created) are flagged P1 and resolved manually by Super Admin.

PART 26 — SEARCH ARCHITECTURE (TYPESENSE)

26.1 Why Typesense Over Algolia

Factor	Typesense	Algolia
Cost at 500 Gaushalas	~₹1,500/month (self-hosted Cloud Run)	~₹3,500/month (Grow plan)
Data sovereignty	Full control, GCP asia-south1	Algolia's servers (not India)
Customization	Full control over ranking	Limited on lower tiers
Hindi/Devanagari support	Supported with ICU tokenizer	Supported
Operational overhead	Low (single binary, Docker)	Zero (fully managed)

Migration path if needed: Typesense Cloud (managed) is available as a zero-ops upgrade if self-hosting becomes a burden.

26.2 Typesense Schema — Gaushalas Collection

```
{
  "name": "gaushalas",
  "fields": [
    { "name": "id", "type": "string" },
    { "name": "name", "type": "string" },
    { "name": "name_hi", "type": "string", "optional": true },
    { "name": "slug", "type": "string" },
    { "name": "description", "type": "string" },
    { "name": "city", "type": "string", "facet": true },
    { "name": "state", "type": "string", "facet": true },
    { "name": "pincode", "type": "string" },
    { "name": "coordinates", "type": "geopoint" },
    { "name": "verified", "type": "bool", "facet": true },
    { "name": "fcraRegistered", "type": "bool", "facet": true },
    { "name": "transparencyScore", "type": "int32", "facet": false },
    { "name": "totalCows", "type": "int32" },
    { "name": "totalRaised", "type": "int64" },
```

```
    { "name": "totalDonors", "type": "int32" },
    { "name": "tags", "type": "string[]", "facet": true },
    { "name": "coverImageUrl", "type": "string", "optional": true },
    { "name": "status", "type": "string", "facet": true },
    { "name": "featured", "type": "bool", "facet": true },
    { "name": "featuredRank", "type": "int32", "optional": true }
  ],
  "default_sorting_field": "transparencyScore",
  "token_separators": ["-", "."],
  "symbols_to_index": ["₹"]
}
```

Tags vocabulary (multi-select filter in Flutter app): `rescue`, `dairy`, `breeding`, `old-age-care`, `gau-shala`, `pinjrapole`, `organic-farm`, `cow-hospital`

26.3 Search Query Configuration

```
// Flutter app search call (via Cloud Function proxy — never direct Typesense API key in app)
const searchParams = {
  q: searchText,
  query_by: 'name,name_hi,description,city,state,tags',
  query_by_weights: '10,10,3,7,6,8',
  filter_by: buildFilterString(filters), // status:=active AND verified:=true
  sort_by: buildSortString(sortOption),  // transparencyScore:desc OR _text_match:desc
  facet_by: 'city,state,tags,verified',
  per_page: 20,
  page: pageNumber
};

// Geo search ("near me"):
const geoParams = {
  ...searchParams,
  filter_by: `coordinates:(${lat},${lng},50 km) && status:=active`,
  sort_by: `_geo_distance(coordinates, ${lat}, ${lng}):asc`
};
```

Cloud Function proxy for search (protects Typesense admin key):

```
// POST /searchGaushalas — authenticated (Firebase Auth JWT) or anonymous
// Extracts search params, calls Typesense with server-side API key
// Never returns more than 20 results per page
// Never exposes Typesense API key to client
```

26.4 Index Synchronization (Firestore → Typesense)

Sync is maintained by a Firestore trigger Cloud Function:

```
// onGaushalaWrite — fires on any create/update to gaushalas/{gaushalId}
export const syncGaushalaToTypesense = onDocumentWritten(
  'gaushalas/{gaushalId}',
  async (event) => {
```

```
    const after = event.data?.after?.data();
    if (!after || after.status !== 'active') {
      // Remove from index if not active
      await
typesenseClient.collections('gaushalas').documents(event.params.gaushalId).de
lete();
      return;
    }
    // Upsert into Typesense
    await typesenseClient.collections('gaushalas').documents().upsert({
      id: event.params.gaushalId,
      name: after.name,
      // ... map all indexed fields
      coordinates: [after.coordinates.latitude, after.coordinates.longitude]
    });
  }
);
```

What is NOT indexed in Typesense:

- Donor data (any field) — absolute prohibition
- Donation amounts of individual donors
- PAN numbers, phone numbers, email addresses
- gaushala_sensitive documents
- Internal admin notes

26.5 Search Analytics (Privacy-Safe)

Typesense's built-in search analytics (popular queries, zero-result queries) are enabled. These logs contain only search terms and result counts — no user identifiers. Popular zero-result queries are reviewed weekly to improve the index or add synonyms (e.g., "gaushala near me" → synonym for geo search trigger).

PART 27 — PDF SERVICE (FORM 10BE CERTIFICATE)

27.1 Architecture

The PDF service runs as a dedicated Cloud Run service (not inside a Cloud Function) because:

- PDF generation (Puppeteer + Handlebars) is CPU-intensive and memory-heavy (256MB–512MB per PDF)
- Cloud Functions have a 540-second timeout and 8GB memory limit — adequate but wasteful for this workload
- Cloud Run scales to zero (cost-effective) and to many instances (handles April 1 surge)
- The service is isolated: a Puppeteer crash cannot affect payment Cloud Functions

```
Pub/Sub message: DONATION_CAPTURED { donationId, userId, gaushalId }
```

```
      ↓
Cloud Function subscriber: checkCertificateEligibility
  - Is Gaushala's 80G still valid? (section80gValidUntil > now)
  - Does donor have PAN on file? (panEncrypted not null)
  - Is donation amount >= ₹2,000? (if yes, PAN is mandatory and cert is
issued)
    (If < ₹2,000 and no PAN: cert still generated but 80G deduction claim may
not apply)
  If eligible:
      ↓
  Calls Cloud Run PDF Service: POST /generate
  { donationId, certificateType: 'single' }
      ↓
Cloud Run PDF Service:
  1. Fetches donation from Firestore (server-to-server, Admin SDK)
  2. Decrypts donor PAN via Cloud KMS
  3. Fetches Gaushala 80G registration number
  4. Populates Form 10BE Handlebars template
  5. Generates PDF via Puppeteer (headless Chrome)
  6. Uploads to Firebase Storage:
certificates/{userId}/{financialYear}/{certId}.pdf
     (Storage rule: readable only by owning userId and Super Admin)
  7. Updates tax_certificates/{certId} with pdfUrl, generatedAt
  8. Returns { pdfUrl, certificateNumber }
      ↓
Cloud Function: notifyCertificateReady
  - FCM push to donor: "Your 80G certificate is ready"
  - SMS: "GauSevak: 80G certificate for FY {year} ready. Download: {link} -
GauSevak"
```

27.2 Form 10BE Template Specification

The Handlebars template (`/pdf-service/templates/form10be.hbs`) must precisely match the Income Tax Department's Form 10BE format as mandated since FY 2021-22. The template includes:

Required fields (verified by CA):

```
1. Name of the donor: {{donorName}}
2. Address of the donor: {{donorAddress}} (optional if not collected)
3. PAN of the donor: {{panLastFour}} (show as XXXXX{{panLastFour}} for
display)
   Full PAN used for the certificate's legal validity — not displayed,
embedded in metadata
4. Amount of donation in figures: ₹{{amountFormatted}}
5. Amount of donation in words: {{amountInWords}} (e.g., "Five Hundred and
One Rupees Only")
6. Nature of donation: Cash / Kind / Otherwise (always "Otherwise" for
digital payments)
7. Whether donation is eligible for 100% or 50% deduction: per Gaushala's 80G
category
8. Name of the donee (Gaushala): {{gaushalName}}
9. PAN of the donee: {{gaushala80gPan}}
10. Address of the donee: {{gaushalAddress}}
```

```
11. 80G registration number: {{section80gNumber}}
12. Financial year of donation: {{financialYear}}
13. Date of donation: {{donationDate}} (DD/MM/YYYY)
14. Certificate serial number: {{certificateNumber}}
15. Authorized signatory name: {{authorizedSignatoryName}}
16. Authorized signatory designation: {{authorizedSignatoryDesignation}}
17. Date of issue: {{issuedAt}} (DD/MM/YYYY)
18. Platform-generated certificate notice: "This certificate is generated by
Gau-Sevak platform
    on behalf of {{gaushalName}}. For filing Form 10BD, Gaushala must submit
this data to ITR."
```

Authorized signatory: The Gaushala Admin sets their name and designation in the admin panel. This is stored in `gaushalas/{id}` as `authorizedSignatoryName` and `authorizedSignatoryDesignation`. This field is **not** a cryptographic digital signature — it is a printed representation. The CA has confirmed this is acceptable for platform-generated certificates as long as the Gaushala files Form 10BD.

Certificate number format: `GS-80G-{FY_SHORT}-{GAUSHALA_CODE}-{SEQUENCE}` Example: `GS-80G-2526-GSH001-000421`

27.3 Annual Consolidated Certificate (April 1 Batch)

`generateAnnual80gBundle` (Cloud Scheduler: April 1, 00:01 IST):

```
// For each unique (userId, gaushalId) pair with captured donations in the
closing FY:
// 1. Aggregate all eligible donations (status=captured, eligible80g=true)
// 2. Call PDF service: POST /generate-consolidated
// 3. Single PDF covering all donations to that Gaushala for the full year
// 4. certificateNumber for consolidated: GS-80G-2526-GSH001-ANNUAL-
{userId_hash}
// 5. tax_certificates record with all donationIds[] linked

// IMPORTANT: Individual certificates issued during the year remain valid.
// The consolidated certificate is ADDITIONAL — it does not supersede
individual ones.
// Donors can use either for filing; the amounts must not be double-counted.
// A notice on the consolidated certificate states: "This covers all
donations listed.
// Do not claim deduction on both this certificate and individual
certificates."
```

Processing is batched: 50 certificates per Cloud Run instance, parallel instances for scale. April 1 processing window: up to 48 hours (all FY certificates delivered by April 3).

27.4 Certificate Tampering Prevention

Each generated PDF:

1. Is stored in Firebase Storage with a content-hash-based filename

2. Has a QR code embedded that links to `gauSevak.in/verify/{certificateNumber}`
3. The verification endpoint (public, no auth) looks up `tax_certificates` and confirms: certificate number is valid, amount matches, Gaushala name matches
4. PDF metadata includes `Producer: GauSevak Platform v2.0` and `CreationDate`

No client-side PDF generation ever. The `pdfUrl` in Firestore is a Firebase Storage signed URL (7-year TTL matching PMLA retention requirement). After 7 years, the URL expires but the file is archived to Coldline Storage, not deleted.

PART 28 — ANALYTICS & REPORTING ARCHITECTURE

28.1 Two-Tier Analytics System

Tier	Tool	Purpose	Who Sees It
Operational	Firebase Analytics + Crashlytics	User behavior, app crashes, funnel	Internal team
Financial / Business	Cloud SQL + Materialized Views	Donations, payouts, reconciliation	Super Admin, Gaushala Admin
Scale (Phase 4)	BigQuery (nightly Cloud SQL export)	Long-term trends, cohort analysis	Internal team

28.2 Firebase Analytics Events (PII-Free)

All events pass through a PII scrubbing middleware before being logged:

```
// PII scrubber — runs before every analytics call
Map<String, dynamic> scrubPii(Map<String, dynamic> params) {
  final forbidden = ['phone', 'email', 'pan', 'name', 'userId', 'donorId'];
  return params..removeWhere((key, _) => forbidden.any((f) =>
key.toLowerCase().contains(f)));
}
```

Tracked events:

```
// Donation funnel
gaushala_viewed          { gaushal_id, source:
'search'|'featured'|'deeplink'|'share' }
donation_amount_selected { gaushal_id, amount_bucket: '0-100'|'101-500'|'501-
1000'|'1000+' }
payment_initiated        { gaushal_id, payment_method, is_recurring }
payment_success          { gaushal_id, payment_method, is_recurring }
payment_failed           { gaushal_id, failure_reason_code, payment_method }
certificate_downloaded   { financial_year }
```

```
// Discovery
search_performed         { query_length, has_geo_filter, result_count }
filter_applied           { filter_type: 'city'|'tag'|'verified' }
near_me_activated        { result_count }

// Engagement
update_viewed            { gaushal_id, has_video }
campaign_viewed          { campaign_id, gaushal_id }
subscription_created     { gaushal_id, frequency }
subscription_cancelled   { reason:
'user_initiated'|'payment_failed'|'gaushala_suspended' }

// App health
app_opened               { from_notification: bool, deep_link: bool }
offline_mode_entered     {}
offline_queue_processed  { item_count }
```

Explicitly NOT tracked:

- Donor identity (userId, phone, name, email, PAN)
- Exact donation amounts (only buckets)
- Gaushala financials in events (only in Cloud SQL)
- Location below city level

28.3 Gaushala Admin Reports

Accessible in React admin panel, backed by Cloud SQL queries:

Available reports:

Report	Data Source	Export Format
Monthly donation summary	`financial_transactions`	CSV, PDF
Donor acquisition trend	`financial_transactions` (count distinct user_id)	Chart, CSV
Donation by category	`donations` (Firestore)	Chart, CSV
Campaign performance	`campaigns` + `donations`	Chart
Payout history	`payouts` (Firestore)	CSV
Form 10BD data export	`financial_transactions` + donor PAN	CSV (restricted — see Part 8.1)

Cloud SQL query example (monthly summary):

```
SELECT
  DATE_TRUNC('month', captured_at) AS month,
  COUNT(*) AS donation_count,
  SUM(amount_paise) / 100 AS gross_amount_inr,
  SUM(platform_fee_paise) / 100 AS platform_fee_inr,
```

```
  SUM(net_to_gaushala_paise) / 100 AS net_amount_inr,
  COUNT(DISTINCT user_id) AS unique_donors
FROM financial_transactions
WHERE gaushala_id = $1
  AND status = 'captured'
  AND captured_at >= $2
  AND captured_at < $3
GROUP BY 1
ORDER BY 1 DESC;
```

28.4 Super Admin Analytics Dashboard

Backed by Cloud SQL materialized views (refreshed nightly):

```
-- Platform-wide daily summary (materialized, refreshed nightly)
CREATE MATERIALIZED VIEW platform_daily_summary AS
SELECT
  DATE(captured_at) AS date,
  COUNT(*) AS total_donations,
  SUM(amount_paise) AS gross_paise,
  SUM(platform_fee_paise) AS fee_paise,
  COUNT(DISTINCT user_id) AS unique_donors,
  COUNT(DISTINCT gaushala_id) AS active_gaushalas,
  AVG(amount_paise) AS avg_donation_paise,
  COUNT(CASE WHEN is_recurring THEN 1 END) AS recurring_donations
FROM financial_transactions
WHERE status = 'captured'
GROUP BY DATE(captured_at);
```

Key dashboard metrics:

- DAU/MAU (from Firebase Analytics, not Cloud SQL — no PII)
- Daily / weekly / monthly donation volume (Cloud SQL)
- Payment success rate by method (Cloud SQL + Razorpay webhook data)
- Top 10 Gaushalas by volume this month
- New Gaushala applications pending KYC
- Open disputes / refund requests
- 80G certificate generation failure rate
- Upcoming KYC document expiries (next 60 days)

28.5 Reconciliation Report (Monthly)

`monthlyReconciliation` Cloud Function (1st of each month, 3 AM IST):

```
// 1. Fetch Razorpay Route settlement report via API for prior month
// 2. Compare with Cloud SQL financial_transactions for same period
// 3. Identify discrepancies:
//    - Transactions in Razorpay but not in Cloud SQL (missed webhooks)
//    - Transactions in Cloud SQL but not in Razorpay settlements (data
integrity issue)
//    - Amount mismatches
```

```
// 4. Store reconciliation_reports/{reportId} in Firestore with discrepancy
list
// 5. If discrepancy count > 0: PagerDuty P2 alert to Super Admin
// 6. Super Admin reviews and marks each discrepancy as resolved with notes
```

PART 29 — CONTENT MODERATION WORKFLOW

29.1 Moderation Queue (Super Admin Panel)

All Gaushala-uploaded content (photos, videos, text updates) enters a moderation queue before being visible to donors. The queue is visible in the Super Admin panel under "Content Review."

```
Gaushala Admin uploads photo/video → update/{updateId} created with
status='pending_moderation'
→ Super Admin sees item in queue (sorted by upload time, oldest first)
→ Super Admin views content (photo inline, video via Cloudflare Stream
preview)
→ Approve: status='approved', moderatedBy, moderatedAt updated
  → FCM to Gaushala Admin: "Your update has been published."
→ Reject: status='rejected', rejectionReason stored
  → FCM to Gaushala Admin: "Your update was rejected: {reason}"
  → If video: Cloud Function calls Cloudflare Stream DELETE to remove the
video immediately
```

29.2 Moderation Rules

The Super Admin follows these documented guidelines (stored in `docs/content-policy.md`):

Auto-reject (no human review needed — flagged by Cloud Vision API pre-check):

- Adult/explicit content detected (Cloud Vision API SafeSearch)
- Violence or gore detected
- Faces that are not bovine/animal (protects donor privacy in Gaushala photos)

Human review required:

- Text updates claiming specific donation amounts ("We received ₹50,000 this month") — fact-check against platform data
- Photos of injured animals (allowed with context; rejected if gratuitously distressing)
- Photos containing identifiable human faces of minors

Auto-approve (future Phase 4 enhancement):

- Photos where Cloud Vision API detects bovine/cow subjects with confidence > 0.9 and no SafeSearch flags

29.3 Cloud Vision API Integration

```
// Pre-upload check (Cloud Function: moderateContent)
const [safeSearchResult] = await
visionClient.safeSearchDetection(imageBuffer);
const annotation = safeSearchResult.safeSearchAnnotation;

if (
  annotation.adult === 'LIKELY' || annotation.adult === 'VERY_LIKELY' ||
  annotation.violence === 'LIKELY' || annotation.violence === 'VERY_LIKELY'
) {
  // Hard reject; do not create the update document
  return { rejected: true, reason: 'content_policy_violation' };
}
// Otherwise: create update document with status='pending_moderation' for
human review
```

Cloud Vision API is called only for images, not videos (too expensive). Videos go directly to human moderation queue.

29.4 Campaign Approval Workflow

Campaigns (`campaigns/{campaignId}`) follow a separate approval workflow:

```
Gaushala Admin creates campaign (status='draft') → submits
(status='pending_approval')
→ Super Admin reviews: title, description, target amount, image, end date
→ Approve: status='active', approvedBy, approvedAt
  → Campaign becomes visible in Flutter app
  → Optional: Super Admin promotes to "Featured" (featuredRank set)
→ Reject: status='rejected', rejectionReason stored
  → Gaushala Admin can edit and resubmit

Campaign auto-expires: Cloud Scheduler daily check
  → If campaign.endDate < now() AND status='active': status='ended'
  → Gaushala admin notified
  → Campaign removed from active listings; accessible via history
```

PART 30 — CORPORATE CSR PORTAL (PHASE 4)

30.1 Product Overview

The Corporate CSR Portal is a white-label React application (separate from the Gaushala Admin panel and Super Admin panel) that allows corporate HR/CSR teams to:

- Set up an employee giving program for one or multiple Gaushalas
- Make bulk donations on behalf of employees
- Receive a consolidated 80G certificate for the company's CSR filing
- Track employee participation and matching contributions

30.2 New Data Model for CSR

```
organizations/{orgId}
  Fields: name, cin (Corporate Identification Number), pan, gstNumber,
          contactName, contactEmail, contactPhone, billingAddress,
          razorpayCustomerId, csr80gEligible (bool), createdAt

csr_donations/{csrDonationId}
  Fields: orgId, gaushalId, amountPaise, donationType:
'bulk'|'employee_matching',
          employeeCount (if bulk), paymentId, status, financialYear,
          consolidatedCertificateId, createdAt

org_employees/{recordId}
  Fields: orgId, employeeEmail, employeeName, optedIn (bool), matchingEnabled
(bool)

csr_certificates/{certId}
  Fields: orgId, gaushalId, donationIds[], financialYear, totalAmountPaise,
          companyPan, companyCin, certificateNumber, pdfUrl, generatedAt
```

30.3 CSR Payment Flow

Corporate donations are large (₹1L–₹50L+ per transaction). They require:

- Razorpay Payment Links or Bank Transfer (NEFT/RTGS) — not UPI (UPI has ₹1L/day limit)
- Invoice generation before payment (corporate procurement requires purchase orders)
- GST invoice from the platform (if platform fee applies)
- 80G certificate in company name (not employee name) for CSR filing

30.4 Legal Note for CSR

CSR under Companies Act Section 135 has specific requirements: only donations to entities registered under Section 8 / Schedule VII qualify. The platform must verify each Gaushala's Schedule VII eligibility before displaying them in the CSR portal. This is an additional verification field in `gaushala_kyc_documents`.

PART 31 — FCRA COMPLIANCE IMPLEMENTATION

31.1 What Counts as a "Foreign Contribution"

Under FCRA 2010 (as amended 2020), a foreign contribution includes:

- Payments from any foreign national (including NRIs on foreign address)
- Payments from foreign-origin cards (even if the donor lives in India)
- Payments via PayPal, Wise, or any foreign-origin payment instrument
- Donations from foreign companies or their Indian subsidiaries (in some interpretations)

The platform's conservative approach: **any payment originating from a non-Indian payment instrument is treated as a potential foreign contribution.**

31.2 Hard Block Implementation

```
// In createRazorpayOrder Cloud Function:
const gaushala = await db.doc(`gaushalas/${gaushalId}`).get();

if (!gaushala.data()?.fcraRegistered) {
  // Create Razorpay order with international payment methods disabled
  const orderOptions = {
    amount: amountPaise,
    currency: 'INR',
    payment_capture: 1,
    // Razorpay method to restrict international cards:
    method: {
      card: true,       // Allow Indian cards
      netbanking: true,
      upi: true,
      wallet: true,
      international: false  // BLOCK international cards
    }
  };
  // Additionally: check donor's last-used payment method in Firestore.
  // If previousPaymentMethod shows a foreign card BIN, reject at this step
too.
}
```

Donor-facing message when blocked: "This Gaushala is not registered for international donations. Please use an Indian payment method, or explore FCRA-registered Gaushalas." (A filtered search for `fcraRegistered: true` is offered.)

31.3 FCRA Separate Account Routing

For FCRA-registered Gaushalas accepting international donations:

```
// createRazorpayOrder — FCRA branch:
const gaushala = await db.doc(`gaushalas/${gaushalId}`).get();
const sensitive = await db.doc(`gaushala_sensitive/${gaushalId}`).get();

// Route to FCRA bank account (different Linked Account ID)
const linkedAccountId = isInternationalPayment
  ? gaushala.data().razorpayFcraLinkedAccountId  // FCRA account
  : gaushala.data().razorpayLinkedAccountId;      // Domestic account

// FCRA donations are tagged separately in financial_transactions
// for the Gaushala's statutory FCRA return filing
```

FCRA Linked Account onboarding requires the Gaushala to provide their FCRA-designated bank account details separately. This is a separate KYC step gated on `fcraRegistered: true`.

31.4 FCRA Annual Return Support

The platform exports, per FCRA-registered Gaushala, a structured annual report of foreign contributions received, formatted for FCRA Annual Return (FC-4) filing. This export is available in the Gaushala Admin panel under Financial → FCRA Report.

PART 32 — DATA DELETION & DPDP ACT 2023 IMPLEMENTATION

32.1 Deletion Request Lifecycle

```
Donor taps "Delete My Account" in Settings → Privacy

Cloud Function: initiateDataDeletion(userId)
  1. Verify JWT matches userId
  2. Check for active subscriptions → cancel all (Razorpay + Firestore)
  3. Check for pending donations (status='created') → expire
  4. Set users/{userId}.accountStatus = 'deletion_requested'
  5. Set users/{userId}.deletionRequestedAt = now()
  6. Send email confirmation: "Your deletion request is received.
     Data will be deleted within 30 days. Financial records are retained
     for 5 years as required by Indian law."
  7. Write to audit_log: DATA_DELETION_REQUESTED

Cloud Scheduler (daily): processDataDeletionRequests
  → Find users with deletionRequestedAt < (now - 30 days)
  → For each: call executeDataDeletion(userId)

executeDataDeletion(userId):
  1. Anonymize users/{userId}:
     name → "Deleted User"
     phone → null (hashed version retained in separate analytics table)
     email → null
     panEncrypted → DELETE (purge from KMS too)
     kycStatus → 'deleted'
     accountStatus → 'deleted'
  2. Anonymize donations/{donationId} where userId = userId:
     userId → 'deleted_user_[hash]'  // Retain for financial integrity; break
PII link
  3. Anonymize subscriptions — same pattern
  4. Anonymize tax_certificates:
     panLastFour → 'XXXX'
     pdfUrl → DELETE file from Firebase Storage
     (Certificate number retained for audit trail)
  5. Purge notifications/{notifId} where userId = userId — DELETE documents
  6. Revoke Firebase Auth account: admin.auth().deleteUser(userId)
  7. Write to audit_log: DATA_DELETION_EXECUTED (with record count
anonymized, not IDs)
  8. Cloud SQL: anonymize financial_transactions where user_id = userId
     (user_id → 'deleted_[hash]'; all financial figures retained for PMLA)
```

32.2 Legal Hold (PMLA Override)

Financial records are subject to a 5-year legal hold under PMLA. The deletion flow does NOT delete financial records — it anonymizes them by breaking the PII link. The financial data (amounts, dates, Gaushala IDs) is retained but cannot be traced back to the individual.

This is compliant with DPDP Act 2023, which explicitly carves out retention obligations under other laws.

The `legal_hold` flag in Cloud SQL:

```
ALTER TABLE financial_transactions ADD COLUMN legal_hold BOOLEAN NOT NULL
DEFAULT TRUE;
-- legal_hold = TRUE means: anonymize PII but do not delete this row
-- Set to FALSE only when PMLA 5-year retention period expires (automated by
Cloud Scheduler)
```

32.3 Right to Access (Data Export)

```
Donor taps "Export My Data" in Settings → Privacy

Cloud Function: exportPersonalData(userId)
  1. Gather all data about userId:
     - Profile (from users/{userId})
     - All donations (from donations collection)
     - All subscriptions
     - All 80G certificates (PDF URLs, not PDFs inline)
     - All notifications (last 90 days)
  2. Generate JSON export file
  3. Upload to Firebase Storage: data_exports/{userId}/{timestamp}.json
     (URL expires in 7 days — one-time download)
  4. FCM + email with download link
  5. Write to audit_log: DATA_EXPORT_REQUESTED
  Rate limit: max 2 exports per month per user (Upstash Redis)
```

PART 33 — RATE LIMITING IMPLEMENTATION (UPSTASH REDIS)

33.1 Rate Limit Rules

All rate limits enforced server-side in Cloud Functions via Upstash Redis (serverless Redis — no connection pooling issues with Cloud Functions).

```
import { Ratelimit } from '@upstash/ratelimit';
import { Redis } from '@upstash/redis';

const redis = new Redis({ url: process.env.UPSTASH_REDIS_URL!, token:
process.env.UPSTASH_REDIS_TOKEN! });

// Different limiters for different endpoints
const limiters = {
```

```
  createOrder: new Ratelimit({
    redis,
    limiter: Ratelimit.slidingWindow(10, '1 m'), // 10 orders per minute per
user
    prefix: 'rl:createOrder'
  }),
  searchGaushalas: new Ratelimit({
    redis,
    limiter: Ratelimit.slidingWindow(60, '1 m'), // 60 searches per minute
per IP
    prefix: 'rl:search'
  }),
  sendCampaignNotif: new Ratelimit({
    redis,
    limiter: Ratelimit.slidingWindow(3, '24 h'), // 3 campaign notifications
per Gaushala per day
    prefix: 'rl:campaign_notif'
  }),
  exportDonorReport: new Ratelimit({
    redis,
    limiter: Ratelimit.slidingWindow(5, '24 h'), // 5 exports per Gaushala
admin per day
    prefix: 'rl:donor_export'
  }),
  otpRequest: new Ratelimit({
    redis,
    limiter: Ratelimit.slidingWindow(5, '1 h'), // 5 OTP requests per phone
per hour
    prefix: 'rl:otp'
  }),
  dataExport: new Ratelimit({
    redis,
    limiter: Ratelimit.slidingWindow(2, '30 d'), // 2 personal data exports
per month
    prefix: 'rl:data_export'
  })
};

// Usage in Cloud Function:
const { success, limit, remaining } = await
limiters.createOrder.limit(userId);
if (!success) {
  throw new HttpsError('resource-exhausted', 'Too many requests. Please try
again later.');
}
```

33.2 Daily Donation Limit (Fraud Control)

Per-donor daily donation limit enforced via Upstash Redis counter:

```
const dailyKey = `daily_donation:${userId}:${getTodayIST()}`; // Resets at
midnight IST
const dailyTotal = await redis.incrby(dailyKey, amountPaise);
await redis.expire(dailyKey, 86400); // Expire after 24 hours
```

```
const maxDaily = platformConfig.maxDonationAmountDaily; // Default: ₹5,00,000
if (dailyTotal > maxDaily) {
  // Flag for KVYK review if just exceeded; block if far exceeded
  if (dailyTotal > maxDaily * 2) {
    throw new HttpsError('resource-exhausted', 'Daily donation limit
exceeded.');
  }
  // Log to audit_log for fraud review
  await logToAudit(userId, 'DAILY_LIMIT_APPROACHED', { dailyTotal, maxDaily
});
}
```

33.3 Webhook Receiver (No Rate Limit — But Signature Required)

The Razorpay webhook receiver endpoint does NOT rate-limit (Razorpay's IPs are legitimate senders and may burst during campaigns). Instead, it only accepts requests with a valid HMAC-SHA256 signature. An invalid signature returns 400 immediately and is logged.

PART 34 — ERROR HANDLING & CIRCUIT BREAKER PATTERNS

34.1 Flutter App — Graceful Degradation

Every screen in the Flutter app must handle three states explicitly (enforced by Riverpod's `AsyncValue` pattern):

```
// Every data-fetching provider returns AsyncValue<T>
final gaushalaProvider = FutureProvider.family<Gaushala, String>((ref, id)
async {
  // 1. Try fresh data from Firestore
  // 2. On error: try Hive cache
  // 3. On cache miss: throw GaushalaNotFoundError
});

// In the widget:
ref.watch(gaushalaProvider(id)).when(
  data: (gaushala) => GaushalaProfileWidget(gaushala),
  loading: () => GaushalaSkeletonLoader(), // Never show empty screen during
load
  error: (err, _) => ErrorWidget(
    message: err is GaushalaNotFoundError
      ? 'This Gaushala is no longer available.'
      : 'Could not load. Check your connection.',
    action: RetryButton(onTap: () => ref.invalidate(gaushalaProvider(id))),
  ),
);
```

Payment flow error handling (zero ambiguity):

```
// Payment screen states (finite state machine):
enum DonationFlowState {
  idle,
  creatingOrder,
  orderCreated,
  paymentInProgress,
  paymentSuccess,
  paymentFailed,
  paymentPending, // Network dropped after payment; polling
  retryable,      // Can create new order
  nonRetryable    // Duplicate, refunded, etc.
}

// If app is killed/crashed between paymentInProgress and confirmation:
// On next app open, check for any donations with status='created' older than
5 min
// → Show "We're checking your last payment..." banner
// → Poll Cloud Function: checkOrderStatus(orderId) for up to 30 seconds
// → If captured: show success; if failed: show retry; if still created: show
pending
```

34.2 Cloud Function Circuit Breaker (Razorpay API)

If Razorpay API calls fail consistently, the platform must not cascade-fail all donations:

```
// Circuit breaker state stored in Upstash Redis
const BREAKER_KEY = 'circuit:razorpay';
const FAILURE_THRESHOLD = 5;    // 5 consecutive failures
const RECOVERY_TIMEOUT = 60000; // 60 seconds in half-open state

async function callRazorpayWithBreaker<T>(fn: () => Promise<T>): Promise<T> {
  const state = await redis.get(BREAKER_KEY);

  if (state === 'open') {
    // Check if recovery timeout has passed
    const openedAt = await redis.get(`${BREAKER_KEY}:openedAt`);
    if (Date.now() - Number(openedAt) < RECOVERY_TIMEOUT) {
      throw new HttpsError('unavailable', 'Payment service temporarily
unavailable. Please try again in a moment.');
    }
    // Half-open: try one request
    await redis.set(BREAKER_KEY, 'half-open');
  }

  try {
    const result = await fn();
    // Success: reset failure count
    await redis.del(`${BREAKER_KEY}:failures`);
    if (state === 'half-open') await redis.set(BREAKER_KEY, 'closed');
    return result;
  } catch (err) {
    const failures = await redis.incr(`${BREAKER_KEY}:failures`);
    if (failures >= FAILURE_THRESHOLD) {
      await redis.set(BREAKER_KEY, 'open');
      await redis.set(`${BREAKER_KEY}:openedAt`, Date.now().toString());
```

```
      // PagerDuty P1 alert
    }
    throw err;
  }
}
```

Donor-facing message when circuit is open: "Our payment provider is experiencing a brief issue. Your donation details are saved. Please try again in 2 minutes." (Order is NOT created; no money is moved; no partial state.)

34.3 Idempotency Keys for All Mutating Operations

Every Cloud Function that mutates state accepts and enforces an idempotency key:

```
// createRazorpayOrder:
// Client generates: idempotencyKey = UUID (stored in Hive for retry)
// Cloud Function:
const existing = await db.collection('donation_intents')
  .where('idempotencyKey', '==', idempotencyKey)
  .where('userId', '==', userId)
  .limit(1).get();

if (!existing.empty) {
  // Return existing order — client got a network error on first attempt
  return { orderId: existing.docs[0].data().razorpayOrderId,
...existing.docs[0].data() };
}
// Otherwise: create new order, store with idempotencyKey
```

PART 35 — WHATSAPP INTEGRATION

35.1 WhatsApp Sharing (No WhatsApp Business API Required)

Sharing uses standard URL scheme deep links — no WhatsApp API subscription needed:

```
// Share donation receipt
Future<void> shareReceipt(Donation donation) async {
  final dynamicLink = await generateDynamicLink('/receipt/${donation.id}');
  final message = Uri.encodeComponent(
    'मैंने ${donation.gaushalName} को ₹${donation.amountFormatted} दान किया! '
    '□ आप भी गौ-सेवा में योगदान करें: $dynamicLink'
  );
  final whatsappUrl = 'https://wa.me/?text=$message';
  await launchUrl(Uri.parse(whatsappUrl), mode:
LaunchMode.externalApplication);
}

// Share Gaushala profile
Future<void> shareGaushala(Gaushala gaushala) async {
  final dynamicLink = await generateDynamicLink('/g/${gaushala.slug}');
  final message = Uri.encodeComponent(
```

```
    '${gaushala.name} — ${gaushala.city} में ${gaushala.totalCows} गायों की सेवा। '
    'दान करें: $dynamicLink'
  );
  // ...
}
```

35.2 80G Certificate PDF Sharing

The 80G certificate PDF (stored in Firebase Storage) must be shareable directly to WhatsApp as a file:

```
// Download PDF to local temp directory → share via share_plus package
Future<void> shareCertificatePdf(TaxCertificate cert) async {
  final file = await downloadToTemp(cert.pdfUrl,
'80G_${cert.certificateNumber}.pdf');
  await Share.shareXFiles(
    [XFile(file.path)],
    text: 'मेरा 80G प्रमाण पत्र — FY ${cert.financialYear}',
  );
}
```

35.3 WhatsApp Business API (Phase 4 — Optional)

For transactional messages (donation receipts, 80G certificates) via WhatsApp instead of SMS, the platform can integrate WhatsApp Business API via a BSP (Business Solution Provider like Interakt or Wati) in Phase 4. This requires:

- Meta Business account verification
- Pre-approved message templates
- Per-message cost (~₹0.30–₹0.50 for utility messages)

Until Phase 4, SMS via MSG91/Kaleyra covers all critical transactional notifications.

PART 36 — LOCALIZATION IMPLEMENTATION DETAILS

36.1 ARB File Structure

```
lib/core/i18n/
├── intl_en.arb    # English (base)
├── intl_hi.arb    # Hindi (Phase 1)
├── intl_gu.arb    # Gujarati (Phase 4)
├── intl_mr.arb    # Marathi (Phase 4)
└── intl_ta.arb    # Tamil (Phase 4)
```

intl_en.arb (excerpt — all strings must have equivalents in hi):

```
{
  "@@locale": "en",
  "donateNow": "Donate Now",
  "@donateNow": { "description": "Primary CTA button on Gaushala profile" },

  "donationSuccessTitle": "Thank You for Your Donation!",
  "donationSuccessSubtitle": "₹{amount} donated to {gaushalanName}",
  "@donationSuccessSubtitle": {
    "placeholders": {
      "amount": { "type": "String" },
      "gaushalanName": { "type": "String" }
    }
  },

  "cowCount": "{count, plural, =1{1 Cow} other{{count} Cows}}",
  "@cowCount": {
    "placeholders": { "count": { "type": "int", "format": "decimalPattern" }
}
  },

  "amountInIndianSystem": "₹{amount}",
  "@amountInIndianSystem": {
    "description": "Format using Indian number system: 1,00,000 not 100,000"
  }
}
```

36.2 Indian Number Formatting

```
// Indian number system (lakh, crore) — NOT Western thousand separators
String formatIndianCurrency(int paise) {
  final rupees = paise ~/ 100;
  if (rupees >= 10000000) {
    final crore = rupees / 10000000;
    return '₹${crore.toStringAsFixed(crore % 1 == 0 ? 0 : 2)} Cr';
  } else if (rupees >= 100000) {
    final lakh = rupees / 100000;
    return '₹${lakh.toStringAsFixed(lakh % 1 == 0 ? 0 : 2)} L';
  } else if (rupees >= 1000) {
    // Format: 1,23,456 (Indian grouping)
    return '₹${_formatWithIndianGrouping(rupees)}';
  }
  return '₹$rupees';
}

String amountInWords(int paise) {
  // Converts amount to words in both English and Hindi
  // Example: 501 → "Five Hundred and One Rupees Only" / "पाँच सौ एक रुपये मात्र"
  // Library: number_to_words_english (or custom implementation for Hindi)
}
```

36.3 Hindi Font & Rendering

```
# pubspec.yaml
flutter:
  fonts:
```

```
  - family: NotoSansDevanagari
    fonts:
      - asset: assets/fonts/NotoSansDevanagari-Regular.ttf
      - asset: assets/fonts/NotoSansDevanagari-Bold.ttf
        weight: 700
  - family: NotoSansTamil  # Phase 4
    fonts:
      - asset: assets/fonts/NotoSansTamil-Regular.ttf
```

Hindi is the default for users with Indian device locale (any Indian language setting). English is default for all others.

36.4 Typesense Multi-Language Search

```
// Gaushala names stored in both English and Hindi in Typesense index:
// name: "Shri Gopal Gaushala"
// name_hi: "श्री गोपाल गौशाला"
// query_by: 'name,name_hi,city,description'
// This allows searching in Hindi script to find Gaushalas
```

PART 37 — APP STORE & PLAY STORE COMPLIANCE

37.1 Play Store Requirements

Requirement	Implementation
Target API level (current minimum: API 34)	`targetSdkVersion 34` in build.gradle
64-bit support	Flutter generates 64-bit APKs by default
Sensitive permissions declaration	Location (near me), Camera (KYC upload), Storage (certificate save)
DATA_SAFETY section	Declare: Phone number, Email, Name, Financial info collected
Donation app category	"Finance" category; select "Charitable giving" subcategory
Billing policy	Platform does not use Google Play Billing (uses Razorpay) — confirm with Play Store policy team
App Signing by Google Play	Enabled (release key managed by Google)
Play Integrity API	Integrated (see Part 7.5)

DATA_SAFETY disclosure (Play Store):

- Data collected: Phone number (required), Email address (required), Name (required), Payment information (processed by Razorpay — not stored by app)
- Data shared: None (PAN shared with tax authorities only)
- Security practices: Data encrypted in transit and at rest; users can request deletion
- Data not collected: Contacts, Messages, Camera/microphone content

37.2 App Store (iOS) Requirements

Requirement	Implementation
Privacy Nutrition Label	Phone number, Email, Name, Financial info
App Tracking Transparency (ATT)	Firebase Analytics uses IDFA — ATT prompt required if tracking cross-app
In-App Purchase guideline	Confirm: Razorpay is a payment processor for charitable donations; Apple 30% commission does NOT apply to charitable donations processed outside IAP (verify with Apple Developer Relations)
App Attest	Integrated (see Part 7.5)
Minimum iOS version	iOS 14 (covers 97%+ of active iPhones in India as of 2025)

App Store review notes (include in review submission): "This app is a charitable donation platform for registered Indian Gaushalas (cow shelters). All payments are processed through Razorpay, a licensed Indian payment aggregator. The app does not use Apple In-App Purchase as it processes charitable donations, which are exempt per App Store guidelines Section 3.1.1."

37.3 Financial App Compliance (Both Stores)

Both Play Store and App Store require additional review steps for financial apps:

- **Play Store:** Complete the Financial Services declaration; provide Razorpay's PA license number
- **App Store:** May require contact from Apple Financial Services review team — budget 2–4 extra weeks for first submission

---

PART 38 — PRE-LAUNCH CHECKLIST (COMPREHENSIVE)

This checklist supersedes the brief checklist in the original document. Every item must be checked off before any production launch. Owner and due date assigned to each item.

38.1 Legal & Compliance (Must Complete Before Any User Accepts Money)

```
☐ Fintech lawyer sign-off: Razorpay marketplace PA structure confirmed
☐ CA sign-off: Form 10BE template approved; Form 10BD export format confirmed
☐ Terms of Service published at gauSevak.in/terms (DPDP Act 2023 compliant)
☐ Privacy Policy published at gauSevak.in/privacy (DPDP Act 2023 compliant)
☐ Grievance Officer designated and published (DPDP Act requires named contact)
```

☐ PMLA AML/CFT policy documented and stored in legal records
☐ FCRA hard-block mechanism tested with international test card
☐ Section 8 / Trust / Society registration verified for at least 1 Gaushala before launch
☐ Razorpay Route merchant agreement signed; charitable marketplace use case confirmed in writing
☐ DLT SMS template registration confirmed for all 4 critical templates
☐ 80G certificate QR code verification endpoint live and tested

38.2 Security (Must Complete Before Beta Launch)

☐ All Firestore security rules tested via automated test suite (100% of defined scenarios pass)
☐ IDOR test: donor A cannot access donor B's certificates or donations
☐ Amount manipulation test: captured amount mismatch → donation marked failed
☐ Webhook replay test: duplicate razorpayEventId silently ignored (idempotency confirmed)
☐ JWT spoofing test: non-Super Admin cannot call processRefund
☐ PAN access audit: every PAN retrieval logged; bulk retrieval alert tested
☐ SSL certificate pinning configured; rotation strategy documented in ops wiki
☐ Play Integrity API integrated and tested on rooted device (payment blocked)
☐ App Attest integrated for iOS
☐ flutter_secure_storage confirmed working on Android API 34 and iOS 14+
☐ Razorpay webhook secret stored in Secret Manager (not in environment variables)
☐ Cloud KMS key for PAN encryption created; rotation schedule configured (quarterly)
☐ No PII in Firebase Analytics events (code review + automated linter check)
☐ Screenshot prevention enabled on payment and certificate screens
☐ Obfuscated Flutter production build tested (crash report still symbolicated via Sentry)

38.3 Payment (Must Complete Before Processing Real Money)

☐ Razorpay production account KYC complete for platform entity
☐ At least 1 Gaushala fully onboarded as Razorpay Linked Account with bank verified (penny drop confirmed)
☐ One-time payment: end-to-end test with production Razorpay (small real transaction)
☐ Webhook test: success webhook received, donation captured, Cloud SQL row written
☐ Webhook test: failure webhook received, donation marked failed, donor notified
☐ Webhook test: duplicate webhook silently ignored (idempotency table checked)
☐ Refund test: initiated by Super Admin, Razorpay refund processed, donor notified
☐ Amount mismatch test: captured ≠ order amount → donation failed, no payout
☐ FCRA block test: international card rejected for non-FCRA Gaushala

☐ Daily limit test: donor exceeds maxDonationAmountDaily → 429 response
☐ Rate limit test: > 10 createOrder calls in 1 minute from same userId → 429
☐ Circuit breaker test: Razorpay mock returns 5xx → circuit opens → donor-friendly message
☐ Razorpay webhook URL registered in Razorpay dashboard (production URL, not localhost)
☐ Razorpay webhook secret configured in Secret Manager (not test secret)

38.4 Infrastructure (Must Complete Before Beta Launch)

☐ Firebase project (production) created in asia-south1 region (data localization)
☐ Cloud SQL created in asia-south1, Multi-AZ enabled, continuous WAL archiving active
☐ Cloud SQL monthly partitions for financial_transactions created for next 24 months
☐ Firebase Storage cross-region replication enabled (certificates and KYC docs)
☐ Secret Manager secrets created: Razorpay key, webhook secret, KMS key reference, Upstash credentials
☐ Upstash Redis instance created (serverless, GCP-compatible region)
☐ Cloudflare Stream account created; upload limits configured
☐ Typesense 3-node HA cluster deployed on Cloud Run (production)
☐ Cloud Run PDF service deployed; Form 10BE template CA-approved version live
☐ Firebase budget alert: ₹10,000/day threshold configured
☐ Cloud SQL storage alert: 80% threshold configured
☐ PagerDuty integration configured for P1/P2 alerts
☐ Sentry project created for Cloud Functions; source maps uploaded
☐ Firebase Crashlytics integrated in Flutter production build
☐ All Cloud Schedulers deployed and verified (list in ops wiki)
☐ Disaster recovery runbook in /docs/runbooks/disaster-recovery.md
☐ SSL certificate for gauSevak.in configured with Cloudflare
☐ assetlinks.json (Android App Links) live at gauSevak.in/.well-known/
☐ apple-app-site-association (iOS Universal Links) live at gauSevak.in/.well-known/

38.5 Mobile App (Must Complete Before Store Submission)

☐ Flutter app tested on: Android 8.0 (API 26), Android 10, Android 13, Android 14
☐ Flutter app tested on: iOS 14, iOS 16, iOS 17
☐ Flutter app tested on budget Android device (2GB RAM, 32GB storage — e.g. Realme C30)
☐ Offline mode tested: cache shows stale data; payment blocked; outbox retries on reconnect
☐ Deep link tested: WhatsApp share link opens correct Gaushala profile in app
☐ FCM push notification tested on background and killed app state
☐ SMS receipt received on Jio number after donation (DLT delivery confirmed)
☐ 80G certificate generated, downloaded as PDF, QR code verified

☐ Hindi language tested: all strings translated, Devanagari font renders correctly
☐ Dark mode tested on both platforms
☐ Screenshot prevention confirmed on payment and certificate screens
☐ Play Store DATA_SAFETY section completed and submitted
☐ App Store Privacy Nutrition Label completed
☐ Release build obfuscated; crash reports still symbolicated
☐ App Store review notes prepared (charitable donation, non-IAP justification)

38.6 Performance (Must Complete Before 5,000 Users)

☐ Load test Phase 1: 500 concurrent createOrder calls; P99 < 2s
☐ Load test Phase 1: 500 concurrent Gaushala profile views; P99 < 1s
☐ Typesense search load test: 200 concurrent searches; P99 < 300ms
☐ Cloud SQL query EXPLAIN ANALYZE run on all critical queries; no sequential scans
☐ All required Firestore composite indexes deployed (from Part 4.4)
☐ Cloudflare CDN cache hit rate > 80% for Gaushala cover images
☐ Cold start test: createRazorpayOrder first-call latency < 3s (minimum instances configured)
☐ 80G PDF generation: single certificate < 10s; confirmed async (not in webhook handler)

PART 39 — KNOWN LIMITATIONS & ACCEPTED TRADEOFFS

These are explicitly documented decisions where a limitation is accepted in exchange for speed, cost, or simplicity. They are not gaps — they are recorded choices.

Limitation	Accepted Because	Revisit Trigger
Firestore used for primary data (not PostgreSQL)	Speed to market; managed; real-time capable	Firebase bill > ₹2L/month OR Firestore limitations hit
Authorized signatory on 80G certificate is printed text, not cryptographic signature	CA-confirmed acceptable; Gaushala files Form 10BD	If IT Department mandates DSC on Form 10BE
RPO of 1 hour (not real-time replication)	Cost and complexity; 1-hour WAL archiving is adequate for this scale	Regulator mandates shorter RPO OR volume exceeds ₹1Cr/day
Cow count is self-reported by Gaushala admin	No scalable alternative exists for physical verification	If fraud cases emerge; consider photo-based ML verification
WhatsApp sharing uses URL scheme, not Business API	Business API adds cost and Meta dependency	User research shows donors prefer in-WhatsApp receipts

Limitation	Accepted Because	Revisit Trigger
Typesense self-hosted (not Algolia managed)	Cost; data sovereignty	Typesense ops burden > 4 hours/month for team
iOS released in Phase 2, not Phase 1	Reduce initial complexity; Android covers 95%+ of India market	n/a — planned
Annual 80G certificate batch takes up to 48 hours	Cloud Run processing cost vs speed	If donors complain or ITD mandates faster delivery
No real-time Gaushala cow count verification	Impractical at scale	Phase 4: explore IoT or photo-based cow count ML
Platform absorbs GST on platform fee	Simpler donor UX	If GST on donations becomes regulatory issue

---

PART 40 — GLOSSARY

Term	Definition in this Document
80G	Section 80G of the Indian Income Tax Act; allows donors to claim tax deduction on donations to registered entities
Form 10BE	Income Tax Department's mandated certificate format for 80G donations (since FY 2021-22)
Form 10BD	Statement filed by donee (Gaushala) with IT Department reporting all 80G-eligible donations received
FCRA	Foreign Contribution Regulation Act — governs receipt of foreign funds by Indian entities
PMLA	Prevention of Money Laundering Act — mandates KYC, AML/CFT policies, and 5-year record retention
DPDP Act 2023	Digital Personal Data Protection Act 2023 — India's primary data privacy law
PA License	RBI Payment Aggregator License — required for platforms that collect money on behalf of third parties
Razorpay Route	Razorpay's marketplace payout product — splits and transfers payments to Linked Accounts
Linked Account	A beneficiary entity (Gaushala) onboarded on Razorpay Route to receive fund transfers
KVYK	Know Your Donor — platform's enhanced due diligence for high-value donors (>₹50,000/year)
Gaushala	A cow shelter/sanctuary; the beneficiary entity in this platform
Pinjrapole	A type of animal sanctuary (also shelters cows) — a subtype of Gaushala in this platform
Transparency	Platform-computed trustworthiness score (0–100) for each Gaushala; see Part

Term	Definition in this Document
Score	18
DLT	Distributed Ledger Technology — TRAI's mandatory SMS sender registration system for Indian telcos
ADR	Architecture Decision Record — documented rationale for a significant technical decision
KMS	Key Management Service (Google Cloud KMS) — used for encrypting PAN and bank account data
paise	1/100th of an Indian Rupee; all monetary values stored as integers in paise to avoid floating-point errors
Outbox Pattern	A reliability pattern where failed operations are queued locally and retried when conditions are met
Idempotency	Property of an operation where executing it multiple times produces the same result as executing it once
Atomic Increment	A database operation that reads and writes a counter in a single indivisible step, preventing race conditions
Circuit Breaker	A pattern that stops calling a failing service after a threshold, allowing it to recover
WAL Archiving	Write-Ahead Log archiving — continuous PostgreSQL backup that enables point-in-time recovery
Sub-merchant	A business onboarded under a licensed Payment Aggregator's umbrella; Gaushalas are sub-merchants here
asia-south1	Google Cloud's Mumbai, India region — used for data localization compliance
NACH	National Automated Clearing House — RBI-mandated system for recurring debit mandates
UPI Autopay	UPI-based recurring payment mandate — simpler than NACH for small recurring donations
Section 12A	IT Act registration giving charitable trusts/societies income tax exemption
Section 8 Company	A company registered under Companies Act for non-profit purposes
Schedule VII	Schedule to Companies Act 2013 listing CSR-eligible activities; includes animal welfare
SLO	Service Level Objective — a measurable target for service reliability (uptime, latency)
RTO	Recovery Time Objective — maximum acceptable downtime after a failure
RPO	Recovery Point Objective — maximum acceptable data loss (in time) after a failure
IDOR	Insecure Direct Object Reference — a security vulnerability where user A can access user B's data
BIN	Bank Identification Number — first 6 digits of a card number, identifies

Term	Definition in this Document
	card's issuing country
Pub/Sub	Google Cloud Pub/Sub — managed message queue used for async event processing
Cloud Run	Google Cloud's serverless container platform — used for PDF generation and Typesense
Cloudflare Stream	Cloudflare's video hosting and transcoding service — used for Gaushala gallery videos

---

END OF GAU-SEVAK MASTER SYSTEM PROMPT

Version: 2.0 Complete (Parts 1–40) **Total Parts:** 40 **Status:** Production-ready specification. No stubs. No "to be determined." Every architectural decision is made, every compliance requirement is addressed, every edge case has a defined handling path.

How to use this prompt with an AI coding assistant: Provide this complete document as the system context. When asking for code, reference the specific Part number (e.g., "Implement the Cloud Function described in Part 5.1, Step 6"). The assistant will have full context for every decision and constraint.

How to use this prompt with an engineering team: Parts 1–3 are for the whole team (legal, product, architecture). Parts 4–9 are for backend engineers. Parts 7, 20, 35–36 are for Flutter engineers. Parts 8, 28–30 are for frontend (React) engineers. Parts 12, 38 are for DevOps. Parts 2, 10, 14, 18, 32 are for compliance and legal review.

GAU-SEVAK — MASTER SYSTEM PROMPT (CONTINUATION & COMPLETION)

IMPORTANT: This document is the direct continuation of the Gau-Sevak Master System Prompt Version 2.0 (Parts 1–40). Append everything below immediately after the Glossary table at the end of Part 40. Do not alter any existing parts. This completion covers Parts 41–60.

PART 41 — TALLY / ZOHO BOOKS INTEGRATION (PHASE 4)

41.1 Why Accounting Integration Is Non-Negotiable at Scale

At ₹10 Cr+ annual donation volume across 500+ Gaushalas, the following tasks exceed the capacity of spreadsheets and manual reconciliation:

- Monthly GST returns (GSTR-1, GSTR-3B) on platform commission income
- TDS certificates for donors (Form 16A where applicable)
- Gaushala-level P&L statements for their annual compliance filings
- Platform's own books: income (commission), expenses (infrastructure, salaries), and tax liability

The platform must offer Gaushalas a structured data export compatible with Tally Prime and Zoho Books. From Phase 3 onwards, the Super Admin panel also needs accounting software integration for platform-level books.

41.2 Tally Prime Integration (Gaushala-Facing)

Tally Prime supports XML-based voucher imports. The platform generates Tally-compatible XML exports per Gaushala on demand:

```
// Cloud Function: generateTallyExport(gaushalId, startDate, endDate)
// Caller: Gaushala Admin (JWT-verified)
// Rate limit: 5 exports per day per Gaushala admin (Upstash Redis)

interface TallyVoucher {
  voucherType: 'Receipt';    // Donations are receipts in Tally
  date: string;              // DD-MMM-YYYY format (Tally standard)
  narration: string;         // "Donation from GauSevak Platform | Receipt
#{receiptNumber}"
  debitLedger: string;       // "Bank Account" (Gaushala's bank account name
in Tally)
```

```
  creditLedger: string;      // "Donations Received" (80G-registered income
ledger)
  amount: number;            // In rupees (not paise) — Tally uses rupees
  gstApplicable: false;      // Charitable donations are GST-exempt
  reference: string;         // Platform receipt number (GS-2025-000001)
}
```

Tally XML structure for each donation:

```
<ENVELOPE>
  <HEADER>
    <TALLYREQUEST>Import Data</TALLYREQUEST>
  </HEADER>
  <BODY>
    <IMPORTDATA>
      <REQUESTDESC>
        <REPORTNAME>Vouchers</REPORTNAME>
      </REQUESTDESC>
      <REQUESTDATA>
        <TALLYMESSAGE xmlns:UDF="TallyUDF">
          <VOUCHER VCHTYPE="Receipt" ACTION="Create">
            <DATE>{{date_DDMMMYYYY}}</DATE>
            <VOUCHERTYPENAME>Receipt</VOUCHERTYPENAME>
            <NARRATION>{{narration}}</NARRATION>
            <ALLLEDGERENTRIES.LIST>
              <LEDGERNAME>{{bank_ledger}}</LEDGERNAME>
              <ISDEEMEDPOSITIVE>Yes</ISDEEMEDPOSITIVE>
              <AMOUNT>-{{amount}}</AMOUNT>
            </ALLLEDGERENTRIES.LIST>
            <ALLLEDGERENTRIES.LIST>
              <LEDGERNAME>{{donation_income_ledger}}</LEDGERNAME>
              <ISDEEMEDPOSITIVE>No</ISDEEMEDPOSITIVE>
              <AMOUNT>{{amount}}</AMOUNT>
            </ALLLEDGERENTRIES.LIST>
          </VOUCHER>
        </TALLYMESSAGE>
      </REQUESTDATA>
    </IMPORTDATA>
  </BODY>
</ENVELOPE>
```

The export is a ZIP file containing one XML file per month in the requested date range. The Gaushala accountant imports each XML file directly into Tally Prime.

41.3 Zoho Books Integration (API-Based)

For Gaushalas using Zoho Books, the platform offers direct API push via OAuth 2.0. The Gaushala Admin connects their Zoho Books organization in the admin panel:

```
1. Gaushala Admin clicks "Connect Zoho Books" in Settings → Integrations
2. OAuth 2.0 flow → Gaushala authorizes read/write on their Zoho Books org
3. Platform stores OAuth tokens in gaushala_sensitive/{id}.zohoTokens
(encrypted via KMS)
```

```
4. Cloud Scheduler (daily 2 AM IST): pushes prior day's donations as
"Customer Payments" in Zoho Books
5. Each donation → one Zoho Books CustomerPayment:
   {
     customer_id: "GauSevak-Platform",  // Fixed vendor for all platform
donations
     payment_mode: "Online",
     amount: amount_inr,
     reference_number: receiptNumber,
     date: capturedAt_YYYY_MM_DD,
     description: "Donation | ${category} | ${donorFirstName} via GauSevak"
   }
```

Token refresh is handled automatically. On OAuth token expiry or revocation, the integration status in the admin panel shows "Disconnected" and a reconnect CTA is shown.

41.4 Platform-Level Accounting (Super Admin)

The platform's own financials are managed via Zoho Books (platform organization). Cloud Function `syncPlatformFeeToZohoBooks` runs daily:

- Revenue: platform commission per donation → Sales Invoice in Zoho Books
- Expense: Razorpay processing fee → Vendor Bill in Zoho Books
- Liability: GST payable → tracked via Zoho Books Tax Summary
- Reconciliation: monthly Cloud SQL `monthly_reconciliation` view exports to CSV, imported into Zoho Books for audit

---

PART 42 — COW SPONSORSHIP & ADOPTION PROGRAM (PHASE 4)

42.1 Product Design

Cow sponsorship allows a donor to "adopt" a specific named cow at a Gaushala, paying a monthly fee for that cow's care. This is distinct from a regular recurring donation in that:

- The sponsorship is tied to a specific cow (named, photographed, numbered)
- The donor receives monthly updates specifically about their sponsored cow
- A "sponsorship certificate" is generated (distinct from 80G certificate)
- The cow has a unique identifier and public profile page

42.2 New Data Models for Sponsorship

```
cows/{cowId}
  Fields: gaushalId, name, tagNumber, breed, gender, age (years),
          healthStatus: enum(healthy | under_care | critical | recovered),
          rescueDate: Timestamp | null,
```

```
          rescueStory: string | null,
          coverImageUrl: string,
          imageUrls: array<string>,
          sponsorshipAvailable: boolean,
          sponsorshipAmountMonthly: number (in paise),
          currentSponsorUserId: string | null,
          currentSponsorshipId: string | null,
          totalSponsorsCount: number,
          createdAt: Timestamp,
          updatedAt: Timestamp

cow_sponsorships/{sponsorshipId}
  Fields: userId, cowId, gaushalId,
          amountMonthly: number (in paise),
          razorpaySubscriptionId: string,
          status: enum(active | paused | cancelled | cow_transferred),
          startDate: Timestamp,
          endDate: Timestamp | null,
          certificateUrl: string | null,
          certificateGeneratedAt: Timestamp | null,
          createdAt: Timestamp

cow_updates/{updateId}
  Fields: cowId, gaushalId, authorId,
          title: string,
          description: string,
          imageUrls: array<string>,
          videoUrl: string | null,
          cowHealthStatus: enum(healthy | under_care | critical | recovered),
          weightKg: number | null,
          status: enum(pending_moderation | approved | rejected),
          targetAudienceType: enum(all_sponsors | specific_sponsor | public),
          targetSponsorUserId: string | null,
          createdAt: Timestamp
```

42.3 Sponsorship Payment Architecture

Cow sponsorship uses the same Razorpay Subscriptions + Route pattern as recurring donations (Part 25), with these differences:

- Plan amount is set per-cow by the Gaushala admin (minimum ₹500/month, maximum ₹5,000/month)
- Each active sponsorship = one active Razorpay Subscription on the platform account
- On each charge: transfer to Gaushala Linked Account (same "transfer after capture" pattern)
- When a sponsored cow is transferred, dies, or is otherwise removed: sponsorship is cancelled with a full explanation to the donor; prorated refund for the unused portion of the current month

What happens when a sponsored cow dies:

```
Cloud Function: markCowDeceased(cowId, date, cause)
  → Caller: Gaushala Admin (JWT-verified admin of that gaushalId)
```

```
  → If currentSponsorUserId exists:
    a. Cancel Razorpay Subscription immediately
    b. cow_sponsorships/{id}.status = 'cancelled', endDate = now()
    c. FCM + SMS + email to donor: compassionate notification template
       (Pre-written; reviewed by a counsellor for sensitivity)
    d. Prorated refund for unused days in current billing cycle
    e. Offer donor: re-sponsor another cow (deep link to Gaushala's
sponsorable cows)
  → cow.sponsorshipAvailable = false
  → Gaushala cow count decremented
  → Gaushala admin prompted to add a memorial update for the cow's profile
```

42.4 Sponsorship Certificate

A sponsorship certificate (distinct from 80G) is generated when sponsorship is activated:

- Certificate includes: donor name, cow name, cow photo, Gaushala name, monthly amount, start date
- Not a tax document — purely a commemorative/engagement artifact
- Generated by the same Cloud Run PDF service using a separate Handlebars template
- Donor can download from their app and share to WhatsApp
- Certificate number format: `GS-SPON-{cowId_short}-{userId_hash}-{year}`

42.5 Cow Profile Page (Public-Facing in Flutter App)

```
// Route: /cow/{cowId}
// Public page — accessible without auth (for sharing)
// Shows: cow name, photo, breed, age, health status, rescue story
// If sponsorshipAvailable: "Sponsor This Cow" CTA
// If sponsored: "This cow is sponsored by [Donor First Name]" (with donor
consent)
//   Donor can toggle their sponsorship visibility in settings
// Cow update feed: approved updates visible to public
// Deep link shareable: "Meet Gopi — sponsored at Shri Gopal Gaushala"
```

PART 43 — PHYSICAL DONATION DRIVES INTEGRATION (PHASE 4)

43.1 What This Feature Covers

Some donors prefer to donate physical items (fodder, medicine, equipment) rather than money. Physical donation drives allow Gaushalas to post specific in-kind needs, and donors to fulfill them. The platform facilitates the coordination but does NOT process payments for physical goods (avoids complex logistics/e-commerce compliance).

43.2 New Data Models

```
donation_drives/{driveId}
```

```
  Fields: gaushalId, title, description,
          driveType: enum(fodder | medicine | equipment | other),
          items: array<{
            name: string,
            quantity: number,
            unit: string (kg | packets | units | litres),
            fulfilledQuantity: number,
            urgency: enum(critical | high | normal)
          }>,
          collectionAddress: string,
          collectionSchedule: string (free text: "Monday–Saturday 9AM–5PM"),
          startDate: Timestamp,
          endDate: Timestamp,
          status: enum(draft | active | fulfilled | ended),
          imageUrl: string | null,
          driveCoordinatorName: string,
          driveCoordinatorPhone: string (hashed for public; shown post-RSVP),
          createdAt: Timestamp

drive_pledges/{pledgeId}
  Fields: driveId, gaushalId, userId,
          pledgedItems: array<{ itemName: string, quantity: number, unit:
string }>,
          deliveryDate: Timestamp | null,
          status: enum(pledged | confirmed | delivered | cancelled),
          donorNotes: string | null,
          createdAt: Timestamp
```

43.3 Platform Role & Liability Limitation

The platform's role in physical drives is limited to:

1. Listing the drive (after moderation approval)
2. Allowing donors to submit pledges (intent to donate)
3. Connecting the donor with the Gaushala coordinator (phone number revealed post-pledge)
4. Tracking fulfillment status (Gaushala admin marks items as delivered)

The platform is explicitly NOT:

- A logistics provider
- Guaranteeing delivery of physical items
- Liable for quality or quantity disputes

ToS includes a specific clause: "Physical donation drives are informational listings. [Platform name] does not verify, facilitate, or guarantee physical donations. Donors and Gaushalas coordinate directly."

80G eligibility for physical donations: Physical donations to 80G-registered entities ARE eligible for tax deduction under Section 80G. However, the platform cannot generate Form 10BE

for physical donations since it cannot verify the fair market value of donated goods. The ToS directs donors to obtain a manual receipt from the Gaushala and consult their CA for valuation.

PART 44 — ACCOUNT RECOVERY FLOW (COMPLETE IMPLEMENTATION)

44.1 Why Account Recovery Is Critical

A donor who changes their phone number loses:

- Access to all 80G certificates (statutory documents)
- Donation history
- Active subscriptions
- All accumulated donation records

This is not a minor UX issue — it is a compliance risk (donor cannot file taxes without their 80G certificates). The recovery flow must be bulletproof.

44.2 Recovery Scenarios & Handling

Scenario A: Donor has email on file (Standard Recovery)

```
1. Donor opens app → "I don't have access to my phone number"
2. Enters registered email address
3. Firebase Auth sends email sign-in link (passwordless)
4. Donor clicks email link → authenticated
5. Prompted to add new phone number (mandatory before accessing financial
features)
6. New phone OTP verified
7. Firebase Auth: old phone credential removed, new phone credential linked
8. users/{userId}.phone updated (hashed) via Cloud Function
9. Audit log: PHONE_NUMBER_CHANGED, ipAddress, timestamp
10. FCM/SMS to old phone (if still reachable): "Your GauSevak account phone
was changed. Contact support if you didn't do this."
```

Scenario B: Donor has no email on file (Support-Assisted)

```
1. Donor contacts support via in-app "Help" → "Account Recovery"
2. Support ticket created with: last-used phone, last 3 donation amounts +
Gaushala names (knowledge auth)
3. Super Admin verifies: checks donations collection against claimed amounts
4. If verified: Super Admin triggers manual recovery email via admin panel
5. Cloud Function: sendRecoveryLink(userId, supportTicketId)
   → Generates time-limited recovery token (1 hour, single-use)
   → Sends email to address provided by donor in support ticket
6. Donor sets new phone number via recovery link
7. Audit log: ACCOUNT_RECOVERED_SUPPORT_ASSISTED
```

Scenario C: New phone AND no email (Extreme Edge Case)

```
1. Support-assisted with enhanced verification:
   - PAN number verification (if on file) — last 4 digits confirmed
   - Transaction ID from donation history (from bank statement)
   - Video KYC call with Super Admin (for donors with >₹10,000 cumulative
donations)
2. If verified: new user account created with same historical data migrated
   (Old account soft-deleted, new account gets all donations/certificates by
reference copy)
3. Compliance note: PAN association is re-verified before any new 80G
certificates are issued
4. Audit log: ACCOUNT_RECOVERED_ENHANCED_KYC
```

44.3 Self-Service Phone Change (In-App, For Users With Account Access)

```
Settings → Account → Change Phone Number
  Step 1: Verify identity — enter current phone OTP
  Step 2: Enter new phone number
  Step 3: Verify new phone — new phone OTP
  Step 4: Firebase Auth: unlink old phone, link new phone
          Cloud Function: updatePhoneHash(userId, newPhoneHash)
  Audit log: PHONE_NUMBER_CHANGED
  Email notification to registered email (if any)
```

PART 45 — FESTIVAL CAMPAIGN SYSTEM (FULL SPECIFICATION)

45.1 Festival Campaign Types

Beyond standard campaigns (Part 4 — campaigns collection), the platform supports platform-wide festival campaigns distinct from Gaushala-specific campaigns:

```
festival_campaigns/{festivalId}
  Fields: name: string (e.g. "Gopashtami 2025")
          nameHi: string ("गोपाष्टमी 2025")
          description: string
          startDate: Timestamp
          endDate: Timestamp
          coverImageUrl: string
          bannerImageUrl: string (hero banner in Flutter app home screen)
          participatingGaushalaIds: array<string>
          totalRaised: number (atomic increment across all participating
Gaushalas)
          totalDonors: number (atomic increment, deduplicated by userId)
          platformGoalAmount: number (platform-wide fundraising goal)
          status: enum(upcoming | active | ended)
          fcmTopicName: string (e.g. "gopashtami_2025" — for opt-in push)
          createdAt: Timestamp
          createdBy: string (Super Admin userId)
```

45.2 Festival Campaign Donation Attribution

When a donor donates to a Gaushala that is part of an active festival campaign, the donation is attributed to both:

1. The Gaushala (normal flow — Gaushala stats updated)
2. The festival campaign (festivalCampaignId on the donation record, festival totalRaised incremented)

```
// Extended donation document fields during festival:
festivalCampaignId: string | null  // If donation happens during active
festival
```

This dual attribution allows the festival leaderboard to show both per-Gaushala and platform-wide totals.

45.3 Festival Leaderboard (Real-Time Feeling)

```
// Flutter: Festival leaderboard uses Firestore real-time listener
// Limited to top 10 Gaushalas to minimize Firestore reads
final leaderboardStream = FirebaseFirestore.instance
  .collection('gaushalas')
  .where('status', isEqualTo: 'active')
  .orderBy('totalRaised', descending: true)
  .limit(10)
  .snapshots();

// Leaderboard refreshes every 60 seconds via stream (not on every donation)
// Cloudflare CDN caches leaderboard API response with 60-second TTL
// Donors see "Updated X seconds ago" label
```

Leaderboard anti-gaming rule: Gaushalas suspended within the last 30 days are excluded from the leaderboard even if their totalRaised is high.

45.4 Festival Surge Preparation Automation

The `prepareFestivalSurge` Cloud Function runs automatically 7 days before any festival campaign's `startDate`:

```
export const prepareFestivalSurge = onSchedule('every day 09:00', async () =>
{
  const upcomingFestivals = await db.collection('festival_campaigns')
    .where('status', '==', 'upcoming')
    .where('startDate', '<=', Timestamp.fromMillis(Date.now() + 7 * 24 * 60 *
60 * 1000))
    .get();

  for (const festival of upcomingFestivals.docs) {
    const participatingGaushalas = festival.data().participatingGaushalaIds;

    // 1. Pre-create distributed counter shards for top Gaushalas
```

```
    for (const gaushalId of participatingGaushalas) {
      await enableDistributedCounters(gaushalId); // 10 shards
    }

    // 2. Alert Super Admin: "Gopashtami 2025 starts in 7 days. Complete
surge prep checklist."
    await sendSuperAdminAlert('FESTIVAL_SURGE_7_DAYS', { festivalName:
festival.data().name });

    // 3. Pre-warm Cloud Functions (set minimum instances)
    // This is done via Firebase console / Terraform — alert triggers manual
action
    // Automated pre-warming via Cloud Functions min-instances update via
Admin SDK would require
    // Firebase Management API — flag for Super Admin to do manually in
Firebase console
  }
});
```

PART 46 — MULTI-ROLE USER SUPPORT

46.1 The Problem With Flat Roles

The original design had a single `role` field on the users document. This breaks for:

- A person who is both a donor AND a Gaushala admin
- A Gaushala admin who manages multiple Gaushalas
- A Gaushala owner who also donates to other Gaushalas

The V2.0 prompt (Part 4) already introduced the `gaushala_admins` junction table and Firebase custom claims for Super Admin. This Part documents the complete role resolution logic to ensure it is implemented consistently.

46.2 Role Resolution Logic (Authoritative)

```
// A user's effective roles are determined as follows — in priority order:

// 1. SUPER ADMIN: Firebase Auth custom claim { superAdmin: true }
//    → Set manually via Firebase Admin SDK; not self-assignable
//    → Has access to all Super Admin panel features
//    → Can also be a donor (dual role — Super Admin donating is allowed and
encouraged)

// 2. GAUSHALA ADMIN: Entry in gaushala_admins/{uid}_{gaushalId}
//    → Can have sub-roles: owner | editor | viewer
//    → owner: full Gaushala admin panel access including financial
//    → editor: can post updates, manage gallery; no financial access
//    → viewer: read-only access to Gaushala dashboard
//    → A user can be admin of multiple Gaushalas (multiple records in
gaushala_admins)
//    → Being a Gaushala admin does NOT restrict donation capabilities
```

```
// 3. DONOR: All authenticated users are donors by default
//    → No special assignment needed
//    → Can donate to any active Gaushala (including their own, if they're
also a Gaushala admin)

// Role derivation in Flutter app:
final authState = ref.watch(authProvider);
final gaushalaAdminships =
ref.watch(gaushalaAdminshipsProvider(authState.userId));
final isSuperAdmin = authState.customClaims['superAdmin'] == true;

// Navigation:
// - isSuperAdmin: show Super Admin panel link in drawer
// - gaushalaAdminships.isNotEmpty: show "My Gaushalas" (admin panel) link in
drawer
// - All users: donor dashboard is default home
```

46.3 Multi-Gaushala Admin UX

When a user is admin of multiple Gaushalas:

```
// Gaushala Admin home screen shows a Gaushala picker:
//   "Shri Gopal Gaushala ▾" (tappable — shows all managed Gaushalas)
//   Switching Gaushala switches the entire admin panel context
//   Selected gaushalId stored in Riverpod state (not persisted — reset on
app restart)

// The gaushala_admins listener:
final gaushalaAdminshipsProvider = StreamProvider.family((ref, userId) {
  return FirebaseFirestore.instance
    .collection('gaushala_admins')
    .where('userId', isEqualTo: userId)
    .snapshots()
    .map((snap) => snap.docs.map((d) =>
GaushalaAdminship.fromDoc(d)).toList());
});
```

PART 47 — GAUSHALA VERIFICATION BADGE SYSTEM

47.1 Badge Tiers (Visible to Donors)

The platform uses a tiered badge system to communicate trust level to donors. Badges are computed by Cloud Function, never self-assigned:

Badge	Visual	Criteria	Meaning to Donor
Verified	✓ Blue shield	All KYC documents approved, not expired	Platform has verified legal registration

Badge	Visual	Criteria	Meaning to Donor
80G Eligible	□ Green leaf	Valid 80G registration on file	Donations are tax-deductible
FCRA Registered	□ Globe	Valid FCRA certificate on file	Accepts international donations
Top Performer	□ Gold star	Transparency Score ≥ 85	Consistently transparent and active
New	□ Sprout	Active for < 90 days	Recently onboarded; less historical data
Needs Renewal	□ Warning	Any KYC document expiring within 30 days	Donor awareness: status may change

Badges are stored as a computed `badges: array<string>` field on the `gaushalas` document, updated by `computeTransparencyScore` daily. A Gaushala cannot have both "Top Performer" and "Needs Renewal" simultaneously — "Needs Renewal" takes precedence and suppresses "Top Performer" until renewal is complete.

47.2 Badge Display Rules in Flutter

```
// Badges are displayed in a horizontal scroll row on the Gaushala profile
header
// Maximum 3 badges shown inline; "+" indicator for more
// Tapping a badge shows a bottom sheet explaining what it means
// "Verified" and "80G Eligible" are always shown first if present

List<String> prioritizedBadges(List<String> badges) {
  final priority = ['needs_renewal', 'verified', '80g_eligible',
'fcra_registered', 'top_performer', 'new'];
  return badges.toList()..sort((a, b) =>
priority.indexOf(a).compareTo(priority.indexOf(b)));
}
```

PART 48 — SEARCH RANKING ALGORITHM (COMPLETE)

48.1 Search Ranking Components

Typesense's default text-match ranking is supplemented by a custom weighted ranking formula. The final ranking score for a Gaushala in search results is:

```
finalScore = (textMatchScore × 0.40)
           + (transparencyScoreNormalized × 0.25)
           + (recentActivityScore × 0.15)
           + (featuredBoost × 0.10)
           + (verifiedBoost × 0.10)
```

```
where:
  textMatchScore         = Typesense's native _text_match score (0–1,
normalized)
  transparencyScoreNorm  = transparencyScore / 100
  recentActivityScore    = min(updatesLast30Days, 10) / 10
  featuredBoost          = featured ? 1 : 0
  verifiedBoost          = verified ? 1 : 0
```

For geo-search ("near me"), textMatchScore is replaced by proximity score:

```
proximityScore = 1 - (distanceKm / 50)  // Linear decay; 0km = 1.0, 50km =
0.0
```

48.2 Preventing Search Gaming

- `featuredBoost` applies only to Gaushalas paying for premium listing (platform_config managed) AND verified
- A Gaushala with `status != 'active'` is excluded from all search results regardless of score
- A Gaushala with `transparencyScore < 20` is de-ranked to bottom regardless of other factors (donor protection)
- `recentActivityScore` counts only `approved` updates (content-moderated), not pending/rejected ones
- Typesense index is write-only from Cloud Functions (Gaushala admins cannot directly update their search index entry)

PART 49 — DONOR COMMUNICATION CONSENT FRAMEWORK

49.1 Consent Types

Under DPDP Act 2023, the platform must obtain and record specific consents. The consent model:

```
user_consents/{consentId}
  Fields: userId,
          consentType: enum(
            platform_terms_and_privacy |       // Mandatory at registration
            80g_data_sharing_with_gaushala |   // Mandatory for 80G (PAN
sharing)
            marketing_communications |         // Optional — app
notifications, emails
            donor_name_on_gaushala_page |      // Optional — "X donors this
month"
            anonymous_analytics |              // Optional — usage analytics
            whatsapp_communications |          // Optional — WhatsApp
receipts
```

```
            sponsor_name_on_cow_profile        // Optional — cow sponsorship
display
          ),
          version: string (e.g. "2025-01-15") // Policy version at time of
consent
          granted: boolean,
          grantedAt: Timestamp | null,
          revokedAt: Timestamp | null,
          ipHash: string,
          userAgent: string
```

49.2 Consent Collection Flow

At registration (mandatory consents):

```
"By continuing, you agree to our Terms of Service and Privacy Policy.
 To receive 80G tax certificates, we share your name and PAN with the
Gaushala
 you donate to, as required by the Income Tax Department."

[ Accept and Continue ]  ← Creates consent records for
platform_terms_and_privacy and 80g_data_sharing_with_gaushala
```

Optional consents (separate, non-blocking):

```
"Enhance your experience:"
[ ✓ ] Receive festival reminders and campaign updates (FCM notifications)
[ ✓ ] Show my name in donor counts on Gaushala pages (shown as "1,234
donors")
[ ] Allow WhatsApp messages for receipts (requires WhatsApp opt-in
separately)
[ ✓ ] Help improve GauSevak with anonymous usage data

[ Save Preferences ]  ← Creates consent records for each toggle
```

49.3 Consent Enforcement in Code

```
// Before any marketing FCM send:
const marketingConsent = await getUserConsent(userId,
'marketing_communications');
if (!marketingConsent?.granted) {
  // Skip this userId in FCM batch; do not send marketing notification
  continue;
}

// Before including donor name in Gaushala's donor count display:
const nameConsent = await getUserConsent(userId,
'donor_name_on_gaushala_page');
// Note: even without name consent, the COUNT is still shown (anonymized
aggregate)
// Only the individual "Rahul donated today" attribution requires consent

// Before generating 80G certificate:
```

```
const sharing80gConsent = await getUserConsent(userId,
'80g_data_sharing_with_gaushala');
if (!sharing80gConsent?.granted) {
  // This should not happen — this consent is mandatory
  // Log as data integrity issue and alert Super Admin
  await logAlert('80G_CONSENT_MISSING', { userId, donationId });
}
```

49.4 Consent Withdrawal

A donor can withdraw any optional consent in Settings → Privacy → Communication Preferences. Withdrawal is immediate:

- FCM marketing sends: donor removed from all marketing topic subscriptions
- Name display: donor's name removed from all Gaushala "recent donors" lists within 24 hours (Cloud Scheduler)
- Analytics: donor's `analyticsId` rotation (new random ID assigned, breaks historical linkage)

Mandatory consents (terms, 80G sharing) cannot be withdrawn while the account is active. If a donor withdraws mandatory consents, they must delete their account (Part 32).

PART 50 — API VERSIONING STRATEGY

50.1 Why API Versioning Is Non-Negotiable

The Flutter app is distributed to millions of devices. Once released, old app versions cannot be force-updated (some users will not update for months). If Cloud Functions change their request/response contract without versioning, old app versions break silently.

50.2 Versioning Scheme

All Cloud Function HTTPS callable endpoints accept an optional `apiVersion` parameter:

```
// Every HTTPS callable Cloud Function:
export const createRazorpayOrder = onCall(async (request) => {
  const apiVersion = request.data.apiVersion ?? 'v1';

  // Version routing:
  if (apiVersion === 'v1') {
    return await handleCreateOrderV1(request);
  } else if (apiVersion === 'v2') {
    return await handleCreateOrderV2(request);
  }
  throw new HttpsError('invalid-argument', 'Unsupported API version');
});
```

Version lifecycle:

- `v1`: Released with Phase 1 app
- `v2`: Released with Phase 3 app (new fields, breaking changes)
- Deprecation: `v1` deprecated (warning in response headers) 90 days after `v2` is live
- Sunset: `v1` removed 180 days after `v2` is live (after force-update enforcement via `min_supported_version` Remote Config)

50.3 Backward Compatibility Rules

For non-breaking changes (adding optional fields, new enum values), no version bump is needed. A version bump is required when:

- Removing a previously accepted field
- Changing the type or meaning of an existing field
- Making a previously optional field mandatory
- Changing error response codes or formats

All API changes are documented in `/docs/api-changelog.md` with the app version that introduced them.

50.4 Flutter App Version Gate

```
// On app start:
final remoteConfig = FirebaseRemoteConfig.instance;
final minVersion = remoteConfig.getString('min_supported_version'); // e.g.
"2.1.0"
final currentVersion = await PackageInfo.fromPlatform().then((p) =>
p.version);

if (isVersionBelow(currentVersion, minVersion)) {
  // Show force update screen — cannot dismiss
  // Opens Play Store / App Store update URL
  // All navigation blocked until updated
  showForceUpdateScreen();
}
```

PART 51 — GAUSHALA CONTENT CALENDAR (ENGAGEMENT TOOLING)

51.1 Purpose

One of the biggest platform risks is "Gaushala ghosting" — a Gaushala stops posting updates, their transparency score drops, donors lose confidence, and donations dry up. The content calendar helps Gaushala admins maintain consistent communication with minimal effort.

51.2 Automated Content Prompts

Cloud Function `sendContentPrompt` runs daily at 9 AM IST and checks each active Gaushala's last update date:

```
If no update in 7 days:
  → FCM to Gaushala admin: "Share a quick update! Your donors haven't heard from you in 7 days.
     Tap to post: [Share cow feeding photo] [Share cow count update] [Share rescue story]"

If no update in 14 days:
  → FCM + email: "Your transparency score has dropped. Post an update today to maintain donor trust."

If no update in 30 days:
  → FCM + email + SMS (urgency): "Your donations have decreased by X% this week.
     Donors trust active Gaushalas. Post an update to re-engage."
  → Super Admin notification: "Gaushala {name} has not posted in 30 days — consider outreach."
```

51.3 Content Templates for Gaushala Admins

The Gaushala admin panel includes a "Quick Post" feature with pre-filled templates in Hindi and English:

```
Template 1: Cow Feeding Update
"आज हमने [__] गायों को [__] kg चारा खिलाया। [Upload Photo]"
"Today we fed [__] cows with [__] kg fodder. [Upload Photo]"

Template 2: New Rescue
"आज हमने एक [नई / घायल / बीमार] गाय को बचाया। [Cow name: __] [Upload Photo]"

Template 3: Monthly Count
"इस महीने हमारे गौशाला में कुल [__] गायें हैं।"

Template 4: Thank You to Donors
"इस महीने [__] दाताओं ने हमें ₹[__] का योगदान दिया। आपका हार्दिक धन्यवाद!"
```

Templates auto-populate where possible (cow count from the last entered value). The Gaushala admin fills in the blanks and taps "Post" — the update goes to content moderation queue (Part 29) before publishing.

PART 52 — PUSH NOTIFICATION DEEP LINK HANDLING (COMPLETE)

52.1 FCM Notification → App Screen Mapping

Every FCM notification sent by the platform includes a `data` payload that maps to a specific app screen. The Flutter app handles these deep links via the GoRouter integration:

```
// In main.dart — FCM message handler:
FirebaseMessaging.onMessageOpenedApp.listen((RemoteMessage message) {
  final data = message.data;
  final action = data['action'];
  final entityId = data['entityId'];

  switch (action) {
    case 'open_donation_receipt':
      context.go('/receipt/$entityId');
      break;
    case 'open_gaushala':
      context.go('/g/${data['slug']}');
      break;
    case 'open_certificate':
      context.go('/certificates/$entityId');
      break;
    case 'open_subscription':
      context.go('/subscriptions/$entityId');
      break;
    case 'open_campaign':
      context.go('/c/$entityId');
      break;
    case 'open_cow_profile':
      context.go('/cow/$entityId');
      break;
    case 'open_refund_status':
      context.go('/history?filter=refunded');
      break;
    default:
      context.go('/notifications');  // Fallback: notification inbox
  }
});

// Handle notification that launched the app from terminated state:
final initialMessage = await FirebaseMessaging.instance.getInitialMessage();
if (initialMessage != null) {
  // Same routing as above — with delay to allow app initialization
  WidgetsBinding.instance.addPostFrameCallback((_) {
    handleNotificationNavigation(initialMessage.data, context);
  });
}
```

52.2 Notification Payload Specification

All FCM notifications sent by Cloud Functions must include both `notification` (for OS tray display) and `data` (for in-app routing):

```
// Standard notification structure for all platform sends:
const message: admin.messaging.Message = {
```

```
  token: deviceFcmToken,
  notification: {
    title: localizedTitle,           // Always localized (hi/en based on
user.language)
    body: localizedBody,
  },
  data: {
    action: 'open_donation_receipt', // Always present
    entityId: donationId,            // The primary entity this notification
is about
    slug: gaushalSlug,               // Secondary reference if needed
    timestamp: Date.now().toString(), // For notification deduplication in-
app
    notifId: notificationDocId,      // For marking as read in Firestore
  },
  android: {
    priority: 'high',                // Required for heads-up notification on
Android
    notification: {
      channelId: getChannelId(notifType), // See channel mapping below
      sound: 'default',
    }
  },
  apns: {
    payload: {
      aps: {
        sound: 'default',
        badge: 1,                    // Increment handled by app on open
        contentAvailable: true,
      }
    }
  }
};
```

Android Notification Channels (must be created in Flutter app on first launch):

Channel ID	Name	Importance	Sound
`payment_critical`	Payment Updates	HIGH	Yes
`certificates`	80G Certificates	DEFAULT	Yes
`subscriptions`	Subscription Alerts	HIGH	Yes
`campaigns`	Campaigns & Festivals	DEFAULT	No
`gaushala_updates`	Gaushala Updates	LOW	No
`system`	System Notifications	DEFAULT	No

PART 53 — SECURITY INCIDENT RESPONSE PLAYBOOK

53.1 Incident Categories & Response SLAs

Category	Example	Response SLA	Escalation
P0 — Critical	Payment data breach, mass unauthorized access	Immediate (< 15 min)	CTO + Legal on call
P1 — High	Webhook processing down, payout failure > 10%	< 30 min	On-call engineer
P2 — Medium	80G certificate generation failing, search down	< 2 hours	Next available engineer
P3 — Low	Non-critical feature bug, single user account issue	< 24 hours	Standard ticket

53.2 Payment Data Breach Response (P0 Playbook)

```
Minute 0–15 (Detection & Containment):
  1. Activate maintenance_mode = true in Firebase Remote Config
     → Flutter app shows maintenance screen; no new donations can be
initiated
  2. Revoke all Super Admin sessions:
admin.auth().revokeRefreshTokens(superAdminUid)
  3. Rotate Razorpay webhook secret in Secret Manager
  4. Rotate Cloud KMS key version (initiates re-encryption of all PAN data)
  5. Alert Razorpay account manager: "Potential compromise — place account
under review"
  6. Document: exactly what was accessed, how many records, time window

Minute 15–60 (Assessment):
  7. Review audit_log for all actions in the suspected breach window
  8. Identify affected userId list (if specific users impacted)
  9. Engage external incident response firm (pre-contracted; details in
/docs/security/incident-response-contacts.md)
  10. Do NOT delete any logs — preserve for forensics

Hour 1–4 (Notification):
  11. DPDP Act 2023: Breach notification to affected individuals required
      — "as soon as possible" (no specific timeframe in DPDP Act, but 72
hours is international standard)
  12. Draft breach notification email (pre-approved template in
/docs/templates/breach-notification-email.txt)
  13. Legal counsel reviews notification before send
  14. Send to affected users via email + SMS (not FCM — FCM may be
compromised)
  15. CERT-In (Computer Emergency Response Team India) notification if
applicable
      (CERT-In mandates reporting within 6 hours for major breaches under IT
Act)

Recovery:
  16. Patch the vulnerability (code review, hotfix deploy)
  17. Penetration test of the specific attack vector
  18. Deactivate maintenance_mode when safe
  19. Post-incident report written within 7 days
  20. Update security controls to prevent recurrence
```

53.3 Suspicious Donation Activity Response

Triggered by: unusual donation patterns (many small donations from one IP, donations to newly-verified Gaushalas spiking suddenly, etc.)

```
// Automated detection (Cloud Function: fraudDetection — runs on each
captured donation):
const indicators = {
  sameIpMultipleDonors: await checkSameIpMultipleDonors(ipHash,
timeWindow1Hour),
  highFrequencyFromUser: userDailyDonationCount > 10,
  newGaushalaHighVolume: isGaushalaActiveLessThan30Days && capturedToday >
50000_00, // ₹50K
  roundNumberPattern: amount % 100_00 === 0 && amount > 1000_00, // ₹1000+
round number
};

const riskScore = Object.values(indicators).filter(Boolean).length;

if (riskScore >= 2) {
  // Flag donation for review
  await db.doc(`donations/${donationId}`).update({ fraudFlag: true,
fraudScore: riskScore });
  await logToAudit(SYSTEM_ID, 'FRAUD_FLAG_RAISED', 'donation', donationId, {
indicators });
  // Super Admin panel shows flagged donations in a dedicated review queue
}
```

53.4 Compromised Gaushala Account Response

If a Gaushala admin account is suspected compromised:

```
1. Super Admin suspends the Gaushala account immediately (status =
'suspended')
   → All active subscriptions paused; payouts held (Part 5.2 flow)
2. Firebase Admin: revokeRefreshTokens(gaushalAdminUserId)
3. Audit log reviewed for all Gaushala admin actions in last 48 hours
4. Contact the Gaushala's registered phone/email (not in-app — account may be
compromised)
5. Re-verify Gaushala admin identity via document KYC before restoring access
6. If fraudulent content was published: remove from search index immediately
   Cloud Function:
typesenseClient.collections('gaushalas').documents(gaushalId).delete()
```

PART 54 — MULTI-LANGUAGE CONTENT MANAGEMENT (GAUSHALA PROFILES)

54.1 The Problem

Gaushala admins in Rajasthan write their descriptions in Hindi. Admins in Tamil Nadu write in Tamil. Donors searching in English see the raw Hindi/Tamil text, which they cannot read. The platform needs a content translation strategy.

54.2 Translation Architecture

```
gaushalas/{gaushalId}/translations/{locale}
  Fields: name: string | null
          description: string | null
          translatedAt: Timestamp
          translatedBy: enum(admin_manual | cloud_translation_api |
ai_generated)
          approvedBy: string | null (Super Admin, if human-reviewed)
          approvedAt: Timestamp | null
```

Translation workflow:

1. Gaushala admin submits content in their primary language
2. Cloud Function `translateGaushalaContent` calls Google Cloud Translation API (automatically)
3. Translation stored in `translations/{locale}` with `translatedBy: 'cloud_translation_api'`
4. Super Admin can review and correct translations (changes `translatedBy: 'admin_manual'`)
5. Flutter app fetches translation for the user's preferred language; falls back to original language if translation unavailable

Cost consideration: Google Cloud Translation API charges ~$20 per million characters. At 500 Gaushalas × average 500 characters description × 3 target languages = 750,000 characters per full refresh = ~$15. Negligible. Translation runs once per content update, not per user request.

54.3 Flutter Locale-Based Content Fetching

```
// Gaushala profile fetches localized content:
Future<GaushalaContent> fetchGaushalaContent(String gaushalId, String locale)
async {
  // Try locale-specific translation first
  final translation = await db
    .collection('gaushalas/$gaushalId/translations')
    .doc(locale)
    .get();

  if (translation.exists && translation.data()?['description'] != null) {
    return GaushalaContent.fromTranslation(translation.data()!);
  }

  // Fall back to primary language (original Gaushala document)
  final primary = await db.doc('gaushalas/$gaushalId').get();
  return GaushalaContent.fromGaushala(primary.data()!);
}
```

PART 55 — SUPPORT SYSTEM INTEGRATION

55.1 In-App Support Flow

The platform does not build a custom ticketing system. It integrates with Freshdesk (or Zoho Desk as an alternative) for support ticket management.

```
support_tickets/{ticketId}
  Fields: userId, userPhone (hashed), category: enum(
            payment_issue | receipt_not_received | refund_request |
            certificate_issue | account_recovery | gaushala_complaint |
            subscription_issue | other
          ),
          description: string,
          status: enum(open | in_progress | resolved | closed),
          freshdeskTicketId: string (reference to Freshdesk ticket),
          attachments: array<string> (Firebase Storage URLs),
          createdAt: Timestamp,
          resolvedAt: Timestamp | null
```

Escalation rules (automated):

Ticket Category	Auto-Escalation Trigger	Action
`payment_issue`	Open > 4 hours	P2 alert to Super Admin
`refund_request`	Open > 24 hours	P2 alert to Super Admin
`certificate_issue`	Open > 48 hours	P3 email to Super Admin
`account_recovery`	Open > 2 hours	P2 alert (time-sensitive)
`gaushala_complaint`	3+ complaints about same Gaushala in 7 days	P1 flag for review

55.2 In-App Help Content

A searchable FAQ is available offline (bundled in app, updated via Remote Config):

```
// FAQ is a JSON file bundled in assets/faq_en.json and assets/faq_hi.json
// Updated via Remote Config (URL to latest FAQ JSON in Firebase Storage)
// Flutter downloads and caches the latest version on app start

// FAQ categories:
// - Donation & Payments
// - 80G Tax Certificates
// - Recurring Donations
// - Account & Profile
// - About GauSevak
// - About Gaushalas
```

WhatsApp support fallback: For complex issues, the in-app "Contact Support" button opens WhatsApp chat with the platform's support number (WhatsApp Business). This requires no API integration — just a `wa.me/{number}?text={pre_filled_message}` URL.

PART 56 — PERFORMANCE BUDGET & FLUTTER APP OPTIMIZATION

56.1 Performance Budget Targets

Metric	Target	Measurement Tool
App cold start (first paint)	< 2 seconds on Realme C30	Firebase Performance Monitoring
Gaushala list load (from cache)	< 300ms	Firebase Performance custom trace
Gaushala list load (from network)	< 1.5 seconds	Firebase Performance custom trace
Donation flow: amount → success screen	< 5 seconds (p99)	Firebase Performance custom trace
Image load (Cloudflare CDN)	< 500ms	Cloudflare Analytics
Search results (Typesense)	< 800ms (p99)	Typesense analytics
80G certificate download	< 3 seconds	Firebase Performance custom trace

56.2 Flutter Performance Optimization Checklist

```
// Image optimization:
// - Use cached_network_image with cacheWidth and cacheHeight set
// - Cloudflare Image Resizing for thumbnails (?width=200&format=webp)
// - Never load full-resolution images in list views

// List performance:
// - Use ListView.builder (not ListView or Column) for all donation/Gaushala
lists
// - Implement pagination (20 items per page) with infinite scroll
// - Never load all Firestore documents — always paginate with
startAfterDocument

// State management performance:
// - Use ref.watch only for reactive UI; ref.read for one-time reads in
callbacks
// - Avoid unnecessary provider rebuilds — use select() to watch specific
fields
// - AutoDisposeProvider for screens — dispose when user navigates away

// Build performance:
// - const constructors everywhere possible
// - Split large widget trees with separate const widgets
// - Profile with flutter DevTools before each release

// Network optimization:
```

```
// - Firestore offline persistence enabled (auto-caching of recently read
documents)
// - Disable persistence for frequently-changing data (donation counters) —
use get() not snapshots()
// - Use snapshots() only for data that benefits from real-time updates
(notifications, subscription status)
```

56.3 Budget Device Testing Protocol

Before every release, test on a minimum-spec device:

Target device: Android phone with 2GB RAM, Snapdragon 439 or equivalent, 32GB storage, Android 8.0 (API 26)

Suggested test device list (₹5,000–₹8,000 range, India market):

- Realme C30
- Tecno Spark 9
- Samsung Galaxy A03

Test scenarios on budget device:

1. App cold start time (fresh install, no cache)
2. Scroll through 50-item Gaushala list — no jank (60fps)
3. Complete donation flow on 3G network (throttled in developer options)
4. Open gallery with 10 images — no OOM crash
5. 80G certificate PDF download — no timeout

PART 57 — ZERO-DOWNTIME DEPLOYMENT STRATEGY

57.1 Firebase Cloud Functions — Blue/Green Deployment

Firebase Cloud Functions do not natively support blue/green deployments. The strategy uses function versioning via Remote Config feature flags:

```
// Two versions of createRazorpayOrder deployed simultaneously:
// createRazorpayOrder_v1 (stable)
// createRazorpayOrder_v2 (new version being rolled out)

// Flutter app reads from Remote Config:
// payment_function_version: "v1" | "v2"

// As confidence in v2 grows:
// Day 0: payment_function_version = "v1" (100% on v1)
```

```
// Day 1: payment_function_version = "v2" for 5% of users (A/B via Remote
Config)
// Day 3: payment_function_version = "v2" for 25%
// Day 7: payment_function_version = "v2" for 100%
// Day 14: Delete v1 function

// If v2 has issues: instantly roll back via Remote Config (no app update
needed)
```

For database schema changes (Cloud SQL migrations), the zero-downtime approach:

1. Deploy additive migration (new nullable column, new table) — no code changes needed yet
2. Deploy new Cloud Function version that writes to both old and new schema
3. Run backfill job to populate new schema from old data
4. Verify backfill completeness
5. Deploy code that reads from new schema only
6. Remove old column/table in a separate migration (after all function versions are updated)

57.2 Firestore Security Rules — Staged Deployment

Firestore security rules deploy atomically (all-or-nothing, no rollback). Therefore:

1. All rules changes must be tested in the Firebase Emulator before staging deployment
2. Rules changes deploy to staging project first (automated CI)
3. Manual QA verifies affected flows in staging
4. Production deployment is a separate manual step with required reviewer approval in GitHub Actions
5. If production rules cause issues: re-deploy previous version (stored in git history)

```
# .github/workflows/deploy-rules.yml
# Manual approval gate for production rules deployment:
- name: Deploy to Production (Requires Approval)
  environment: production  # GitHub environment with required reviewers
configured
  run: firebase deploy --only firestore:rules,firestore:indexes --project
gau-sevak-prod
```

PART 58 — DATA ARCHITECTURE FOR ANALYTICS AT SCALE (BIGQUERY — PHASE 4)

58.1 BigQuery Export Pipeline

At 100K+ users, Cloud SQL materialized views become slow for analytical queries. The migration plan (Part 11, Phase 4) includes BigQuery for analytical workloads.

Export pipeline (Cloud SQL → BigQuery):

```
Cloud Scheduler (daily 3 AM IST): exportToBigQuery
  → Cloud Function reads Cloud SQL: SELECT * FROM financial_transactions
WHERE DATE(created_at) = yesterday
  → Streams to BigQuery dataset: gau_sevak_analytics.financial_transactions
  → BigQuery table partitioned by DATE(created_at) (matches Cloud SQL
partitioning)
  → Retention policy: 3 years in BigQuery standard storage (historical
analysis)
```

BigQuery tables:

```
-- financial_transactions (daily export from Cloud SQL)
-- Anonymized: user_id replaced with user_hash (consistent but non-
reversible)

-- gaushala_daily_stats (generated by computeTransparencyScore Cloud
Function)
-- One row per gaushala_id per day: donations_count, total_raised,
new_donors, transparency_score

-- search_analytics (exported from Typesense)
-- Popular queries, zero-result queries, conversion rate per search query

-- app_events (exported from Firebase Analytics)
-- Already in BigQuery via Firebase Analytics BigQuery export (free feature)
```

58.2 Key BigQuery Analytical Queries

```
-- Donor cohort retention (month-over-month)
WITH first_donations AS (
  SELECT user_hash, DATE_TRUNC(MIN(created_at), MONTH) AS cohort_month
  FROM `gau_sevak_analytics.financial_transactions`
  WHERE status = 'captured'
  GROUP BY user_hash
),
monthly_activity AS (
  SELECT user_hash, DATE_TRUNC(created_at, MONTH) AS activity_month
  FROM `gau_sevak_analytics.financial_transactions`
  WHERE status = 'captured'
  GROUP BY user_hash, activity_month
)
SELECT
  fd.cohort_month,
  ma.activity_month,
  DATE_DIFF(ma.activity_month, fd.cohort_month, MONTH) AS
months_since_cohort,
  COUNT(DISTINCT ma.user_hash) AS retained_donors,
  COUNT(DISTINCT fd.user_hash) AS cohort_size,
  COUNT(DISTINCT ma.user_hash) / COUNT(DISTINCT fd.user_hash) AS
retention_rate
FROM first_donations fd
LEFT JOIN monthly_activity ma ON fd.user_hash = ma.user_hash
GROUP BY 1, 2, 3;

-- Gaushala performance by category (rescue vs dairy vs old-age)
```

```
SELECT
  g.tags,
  COUNT(DISTINCT ft.gaushala_id) AS gaushala_count,
  SUM(ft.amount_paise) / 100 AS total_raised_inr,
  AVG(ft.amount_paise) / 100 AS avg_donation_inr,
  COUNT(*) AS donation_count
FROM `gau_sevak_analytics.financial_transactions` ft
JOIN `gau_sevak_analytics.gaushala_daily_stats` g ON ft.gaushala_id =
g.gaushala_id
WHERE ft.status = 'captured'
  AND DATE(ft.created_at) >= DATE_SUB(CURRENT_DATE(), INTERVAL 90 DAY)
GROUP BY g.tags
ORDER BY total_raised_inr DESC;
```

PART 59 — COMPLETE ENVIRONMENT CONFIGURATION REFERENCE

59.1 Required Environment Variables (All Environments)

All secrets stored in Google Secret Manager. Cloud Functions access via `defineSecret()` (Firebase Functions v2):

```
// functions/src/config/secrets.ts
import { defineSecret } from 'firebase-functions/params';

export const RAZORPAY_KEY_ID          = defineSecret('RAZORPAY_KEY_ID');
export const RAZORPAY_KEY_SECRET      = defineSecret('RAZORPAY_KEY_SECRET');
export const RAZORPAY_WEBHOOK_SECRET  =
defineSecret('RAZORPAY_WEBHOOK_SECRET');
export const UPSTASH_REDIS_URL        = defineSecret('UPSTASH_REDIS_URL');
export const UPSTASH_REDIS_TOKEN      = defineSecret('UPSTASH_REDIS_TOKEN');
export const MSG91_AUTH_KEY           = defineSecret('MSG91_AUTH_KEY');
export const MSG91_TEMPLATE_IDS       = defineSecret('MSG91_TEMPLATE_IDS');
export const CLOUDFLARE_STREAM_TOKEN  =
defineSecret('CLOUDFLARE_STREAM_TOKEN');
export const CLOUDFLARE_ACCOUNT_ID    =
defineSecret('CLOUDFLARE_ACCOUNT_ID');
export const TYPESENSE_ADMIN_API_KEY  =
defineSecret('TYPESENSE_ADMIN_API_KEY');
export const TYPESENSE_SEARCH_API_KEY =
defineSecret('TYPESENSE_SEARCH_API_KEY');
export const TYPESENSE_HOST           = defineSecret('TYPESENSE_HOST');
export const CLOUD_SQL_CONNECTION     =
defineSecret('CLOUD_SQL_CONNECTION_NAME');
export const CLOUD_SQL_PASSWORD       = defineSecret('CLOUD_SQL_PASSWORD');
export const PDF_SERVICE_URL          = defineSecret('PDF_SERVICE_URL');  //
Cloud Run URL
export const KMS_KEY_NAME             = defineSecret('KMS_KEY_NAME');     //
Full KMS key resource name
export const ZOHO_CLIENT_ID           = defineSecret('ZOHO_CLIENT_ID');
export const ZOHO_CLIENT_SECRET       = defineSecret('ZOHO_CLIENT_SECRET');
```

```
export const PAGERDUTY_ROUTING_KEY    =
defineSecret('PAGERDUTY_ROUTING_KEY');
export const CLOUD_VISION_API_KEY     = defineSecret('CLOUD_VISION_API_KEY');
export const FRESHDESK_API_KEY        = defineSecret('FRESHDESK_API_KEY');
```

59.2 Firebase Project Configuration (Per Environment)

Config Key	Development	Staging	Production
Firestore region	`us-east1` (default)	`asia-south1`	`asia-south1`
Firebase Auth region	Default	Default	Default
Cloud Functions region	`asia-south1`	`asia-south1`	`asia-south1`
Razorpay mode	Test	Test	Production
Typesense cluster	Single node local	Single node Cloud Run	3-node HA Cloud Run
Cloud SQL	Emulator / local PG	`db-g1-small`	`db-standard-2` (Multi-AZ)
Min function instances	0	0	5 (for payment functions)
Firebase Storage	Default bucket	`gau-sevak-stg.appspot.com`	`gau-sevak-prod.appspot.com`
SMS provider	Disabled	Test mode (MSG91)	Production (MSG91)
PagerDuty alerts	Disabled	Disabled	Enabled

59.3 Terraform-Managed Infrastructure

All GCP infrastructure is managed via Terraform (in `/infra/terraform/`). Key resources:

```
# Cloud SQL (production)
resource "google_sql_database_instance" "main" {
  name             = "gau-sevak-prod"
  database_version = "POSTGRES_15"
  region           = "asia-south1"

  settings {
    tier              = "db-standard-2"
    availability_type = "REGIONAL"  # Multi-AZ

    backup_configuration {
      enabled                        = true
      start_time                     = "02:00"
      point_in_time_recovery_enabled = true  # WAL archiving for < 1-hour RPO
      transaction_log_retention_days = 7
      retained_backups               = 30
    }

    ip_configuration {
```

```
      ipv4_enabled    = false
      private_network = google_compute_network.main.id
    }
  }
}

# Cloud Run — PDF service
resource "google_cloud_run_service" "pdf_service" {
  name     = "gau-sevak-pdf"
  location = "asia-south1"

  template {
    spec {
      containers {
        image  = "gcr.io/${var.project_id}/pdf-service:latest"
        resources {
          limits   = { cpu = "1", memory = "512Mi" }
          requests = { cpu = "0.25", memory = "256Mi" }
        }
      }
    }
    metadata {
      annotations = {
        "autoscaling.knative.dev/minScale" = "0"
        "autoscaling.knative.dev/maxScale" = "20"
      }
    }
  }
}

# Cloud Run — Typesense (production, 3 nodes managed separately)
# Typesense HA requires 3 separate Cloud Run services with inter-node
discovery
# See /infra/terraform/typesense/ for complete HA configuration
```

PART 60 — FINAL PRODUCTION READINESS MATRIX

This matrix supersedes the score in Document 2 (Section 8). It reflects the complete V2.0 specification after all gaps were addressed.

60.1 Architecture Score: 9/10

Component	Status	Score
Payment architecture (async webhook queue, idempotency)	Fully designed	✓
Database dual-store (Firestore + Cloud SQL)	Fully designed	✓
Video pipeline (Cloudflare Stream)	Fully designed	✓
PDF service (Cloud Run, async)	Fully designed	✓

Component	Status	Score
Search (Typesense, self-hosted)	Fully designed	✓
Scalability migration path (0 → 1M users)	Fully documented	✓
Rate limiting (Upstash Redis)	Fully implemented	✓
Circuit breaker pattern	Fully implemented	✓
Offline support (outbox pattern)	Fully implemented	✓
State management (Riverpod)	Architecture defined	✓
Admin panels (React, decision made)	Architecture defined	✓

Remaining 1 point deducted: Typesense HA operational complexity requires ongoing DevOps attention not fully mitigated by tooling alone.

60.2 Security Score: 9/10

Control	Status
Webhook signature + timestamp validation	✓
IDOR prevention (userId from JWT only)	✓
PAN encryption (Cloud KMS, separate collection)	✓
Audit log (immutable, append-only)	✓
PAN bulk access alerting	✓
Session revocation (Firebase Admin)	✓
SSL pinning with rotation strategy	✓
Play Integrity API + App Attest	✓
Mass assignment prevention	✓
Firestore RLS fully tested	✓
No PII in analytics (scrubbing middleware)	✓
FCRA hard block	✓
UPI direct donation removed	✓

Remaining 1 point: Production penetration test not yet run (by design — occurs in Phase 3).

60.3 Scalability Score: 9/10

Concern	Resolution
Firestore counter race condition	Atomic increments + distributed shards
Connection pool exhaustion	Upstash Redis, Cloud SQL read replicas in Phase 3
Storage egress cost	Cloudflare CDN proxied; Cloudflare Stream for video

Concern	Resolution
RLS performance at scale	Firestore rules (not PostgreSQL RLS) — no per-query overhead
FCM batch sends	sendMulticast (500/batch) in loops
Webhook surge handling	Async queue + Cloud Pub/Sub
1M user migration path	Fully documented with trigger thresholds
Table partitioning	Cloud SQL monthly partitions, auto-created
Analytical query separation	BigQuery for Phase 4 analytics

Remaining 1 point: 1M-user migration executes over 6 months; there is inherent execution risk that cannot be fully mitigated at the design stage.

60.4 Compliance Score: 10/10

Requirement	Status
RBI PA license (Razorpay sub-merchant structure)	✓ Legally confirmed
PMLA (5-year retention, AML/CFT policy)	✓
Section 80G / Form 10BE (CA-reviewed format)	✓
Form 10BD data export for Gaushala	✓
FCRA hard block + separate FCRA account routing	✓
DPDP Act 2023 (consent, deletion, access, localization)	✓
GST on platform fee	✓
DLT SMS registration	✓
KYC document expiry monitoring	✓
Double 80G issuance prevention	✓
Donor data export consent framework	✓
Breach notification process	✓

60.5 Reliability Score: 9/10

Concern	Resolution
RPO 24 hours → 1 hour	WAL archiving + pgBackRest
RTO 4 hours → 2 hours	Practiced DR drill; runbook
Synchronous webhook handling	Fully async queue pattern
No idempotency → idempotency everywhere	webhook_events table + Cloud SQL UNIQUE constraint
No graceful degradation → defined	Riverpod AsyncValue on all screens; offline mode
Circuit breaker for Razorpay	Implemented via Upstash Redis
Festival surge preparation	Automated 7-day pre-surge function

Concern	Resolution
Zero-downtime deployments	Blue/green via Remote Config feature flags
Backup strategy	Firestore + Cloud SQL + Firebase Storage cross-region
DR drill	Quarterly; runbook in /docs/runbooks/

Remaining 1 point: Some Razorpay-specific failure modes (bank NEFT down, Razorpay settlement delays) are external dependencies with no platform-level mitigation beyond alerting.

60.6 Maintainability Score: 9/10

Concern	Resolution
Admin panel decision deferred → React (decided)	✓ ADR-002 written
State management undefined → Riverpod	✓ ADR-001 written
No API versioning → versioned endpoints	✓ Part 50
No coding standards → TypeScript strict mode	✓ tsconfig.json in repo
No migration strategy → Flyway	✓ Part 12.3
No feature flags → Firebase Remote Config	✓ Part 12.5
Monorepo structure defined	✓ Part 12.1
ADR template and required ADRs defined	✓ Part 23
Runbook documentation required	✓ /docs/runbooks/
On-call rotation	✓ PagerDuty

Remaining 1 point: The eventual Supabase → AWS migration (now GCP Cloud Run migration) involves inherent complexity that will require a dedicated 6-month engineering effort — acceptable execution risk, not a design flaw.

END OF GAU-SEVAK MASTER SYSTEM PROMPT

Version: 2.0 Complete + Extension (Parts 1–60) **Total Parts:** 60 **Status:** Production-ready specification. All gaps from Document 2 (critique) are addressed. All stubs resolved. All architectural decisions are made and recorded as ADRs. All compliance requirements have implementation paths. All scale targets have defined migration triggers.

HOW TO USE THIS COMPLETE DOCUMENT

With an AI Coding Assistant (Claude Code, Cursor, etc.)

Provide the complete document (Parts 1–60) as system context. Reference specific Part numbers when requesting code:

- "Implement the Cloud Function described in Part 5.1, Steps 6–7"
- "Build the Flutter donation flow following the state machine in Part 34.1"
- "Write the Firestore security rules from Part 4.3"
- "Create the Cloud SQL schema from Part 4.2 including all indexes"

With an Engineering Team

Team	Read These Parts
All engineers (kickoff)	Parts 1–3, 15, 39, 40
Backend engineers	Parts 4–6, 14, 25, 26, 33, 34, 43, 50
Flutter engineers	Parts 7, 20, 35, 36, 44, 52, 53, 56
React admin engineers	Parts 8, 28, 29, 30, 46
DevOps engineers	Parts 12, 38, 57, 59
Compliance & legal reviewers	Parts 2, 10, 14, 18, 31, 32, 42, 43, 49
Finance & accounting	Parts 16, 28, 41, 58
Product managers	Parts 1, 15, 23, 24, 27, 39, 40

Deployment Sequence (Never Deviate)

1. Complete all Stage 0 checklist items (Part 38.1) before writing Phase 1 code
2. Deploy schema migrations (Part 4.2 Cloud SQL) before any Cloud Function code
3. Test Firestore security rules (Part 4.3) in emulator before staging deploy
4. Complete payment integration testing checklist (Part 38.3) before accepting real money
5. Run load tests (Part 19) before each Phase launch

Document Maintenance

- When architecture decisions change: update the relevant Part AND create/update the ADR in /docs/adr/
- When compliance requirements change: update Part 2 and the relevant implementation Part
- When a new Cloud Function is added: add it to the table in Part 6
- When schema changes: update Part 4 AND write the Flyway migration script

This document is the single source of truth for the Gau-Sevak platform. No architectural decision should exist outside of this document.